

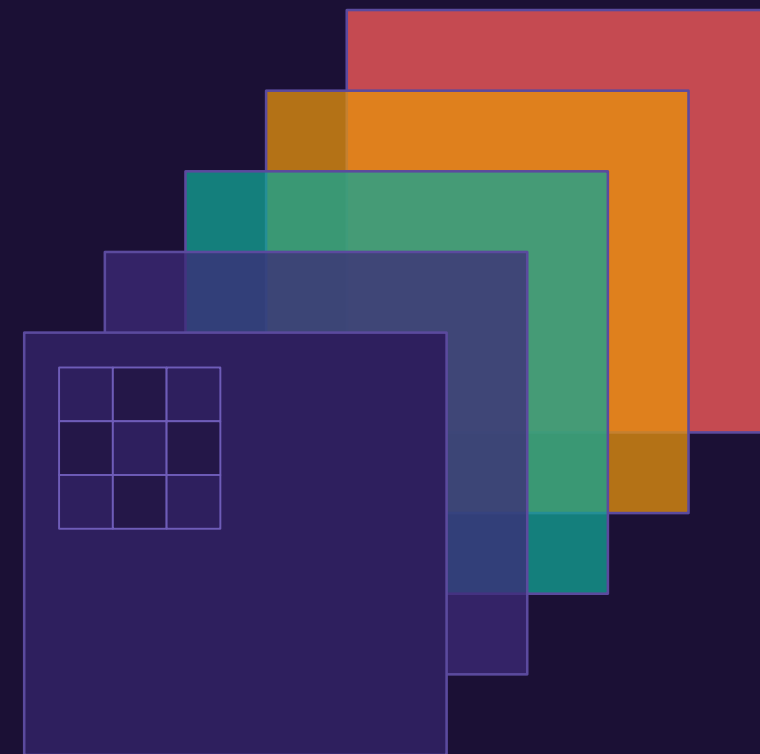
DEEP LEARNING · SESSION 2

# Convolutional Neural Networks

From the convolution operation to the architectures that power modern computer vision

---

Based on Goodfellow, Bengio & Courville (2016) and Stanford CS231n





# What we will cover today

01

## The convolution operation

Kernels, feature maps, non-linearity and multi-channel filters

02

## Why convolution works

Sparse interactions, parameter sharing, equivariance

03

## Anatomy of a conv layer

Stride, padding, pooling and counting parameters

04

## Landmark architectures

LeNet, AlexNet, VGG, GoogLeNet, ResNet, transfer learning

05

## Beyond classification

Detection, semantic and instance segmentation



# 01

## The convolution operation

- How a kernel slides over a grid to build a feature map
- Why we apply an element-wise non-linearity afterwards
- From a single filter to a full 3D output volume

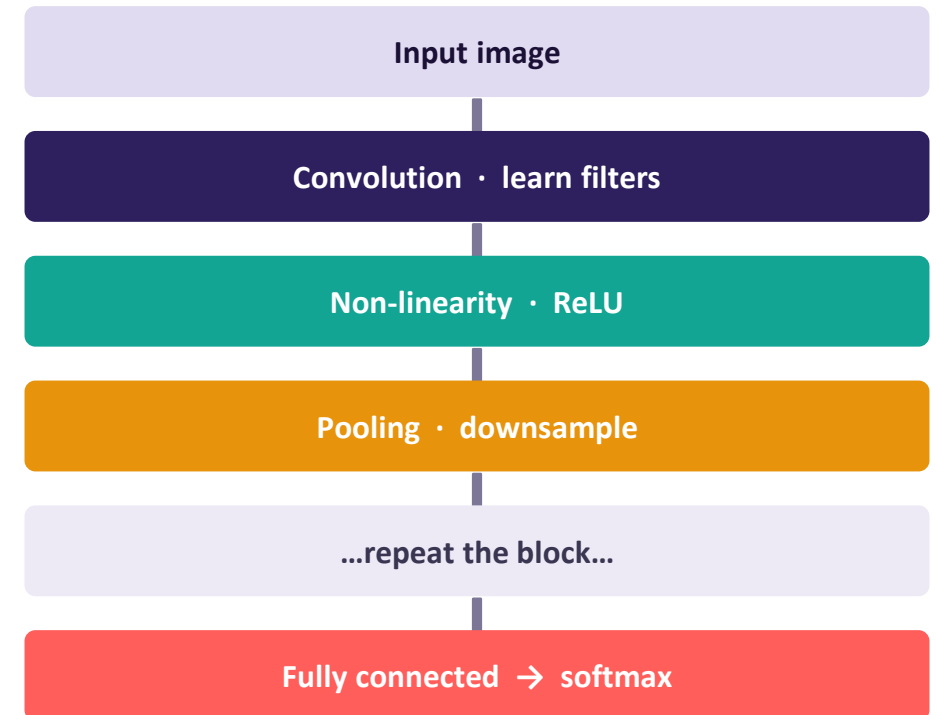


# What is a convolutional neural network?

*A specialised kind of neural network for processing data with a known, grid-like topology.*

- Grid data: images (2D grid of pixels), time series (1D grid of samples), video, audio spectrograms
- Convolutional networks use convolution in place of general matrix multiplication in at least one of their layers
- The kernel is a small set of repeated weights that is reused everywhere in the input

## A typical CNN pipeline





# 2D convolution: how it works

- Slide the kernel over the input, one step at a time — the step size is the stride
- At every position, multiply element-wise and sum the products
- Each sum becomes one cell of the output, called the feature map or convolved feature
- The kernel weights are shared: the same 9 numbers are used at every position

Input image (5 × 5)

|   |   |   |   |   |
|---|---|---|---|---|
| 1 | 1 | 1 | 0 | 0 |
| 0 | 1 | 1 | 1 | 0 |
| 0 | 0 | 1 | 1 | 1 |
| 0 | 0 | 1 | 1 | 0 |
| 0 | 1 | 1 | 0 | 0 |

Kernel (3 × 3)

|   |   |   |
|---|---|---|
| 1 | 0 | 1 |
| 0 | 1 | 0 |
| 1 | 0 | 1 |



Feature map (3 × 3)

|   |   |   |
|---|---|---|
| 4 | 3 | 4 |
| 2 | 4 | 3 |
| 2 | 3 | 4 |

$$1 \cdot 1 + 1 \cdot 0 + 1 \cdot 1 + 0 \cdot 0 + 1 \cdot 1 + 1 \cdot 0 + 0 \cdot 1 + 0 \cdot 0 + 1 \cdot 1 = 4$$

*Output cell =  $\Sigma$  (input patch  $\times$  kernel), element by element*



# Non-linearity, and what filters actually detect

## Element-wise non-linearity

After obtaining a feature map we apply an element-wise non-linear transformation — usually ReLU — to obtain the transformed feature map. Without it, stacking convolutions would collapse into a single linear operation.

Feature map



ReLU:  $\max(0, x)$

Negative activations are clipped to zero, so each filter reports only where its pattern is present — never how strongly it is absent.

## The same operation, different weights

|   |   |   |
|---|---|---|
| 0 | 0 | 0 |
| 0 | 1 | 0 |
| 0 | 0 | 0 |

● Identity

|    |   |    |
|----|---|----|
| 1  | 0 | -1 |
| 0  | 0 | 0  |
| -1 | 0 | 1  |

● Edge detection

|    |    |    |
|----|----|----|
| 0  | -1 | 0  |
| -1 | 5  | -1 |
| 0  | -1 | 0  |

● Sharpen

|   |   |   |
|---|---|---|
| 1 | 1 | 1 |
| 1 | 1 | 1 |
| 1 | 1 | 1 |

● Box blur ( $\times 1/9$ )

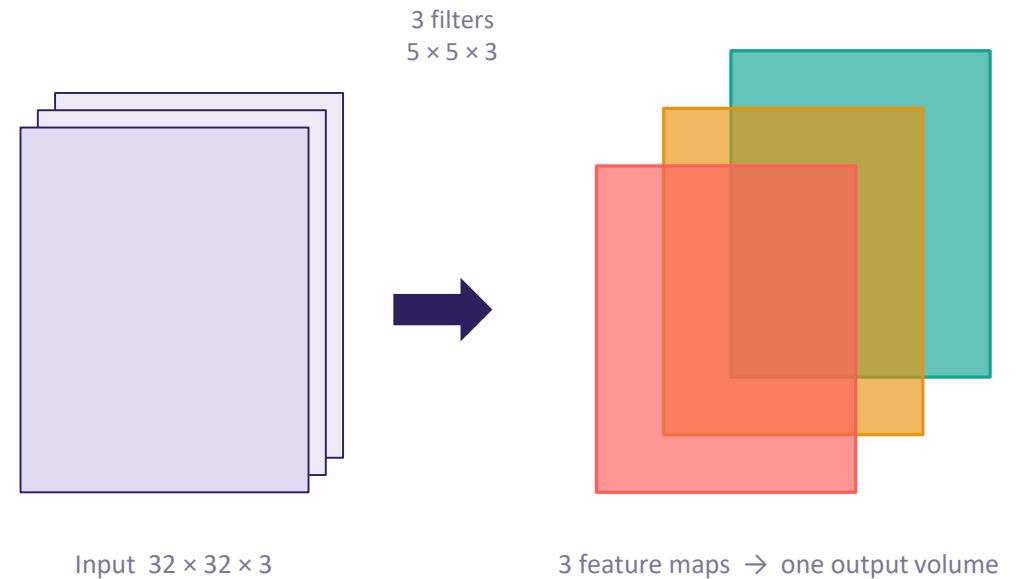


# One filter, one feature map — so we use many

- A single kernel detects a single pattern. Using  $K$  filters produces  $K$  feature maps, stacked into an output volume.
- Filters always span the full depth of their input: for an RGB image a  $5 \times 5$  filter is really  $5 \times 5 \times 3$ .
- The filter slides only in width and height — never in depth — so the output of one filter is always 2-D.
- Output volume =  $W' \times H' \times K$ , where  $K$  is the number of filters, also called the kernel depth.

*Early layers learn edges and colour blobs; deeper layers combine them into textures, parts and objects.*

Using 3 filters produces 3 feature maps





# 02

## Why convolution works

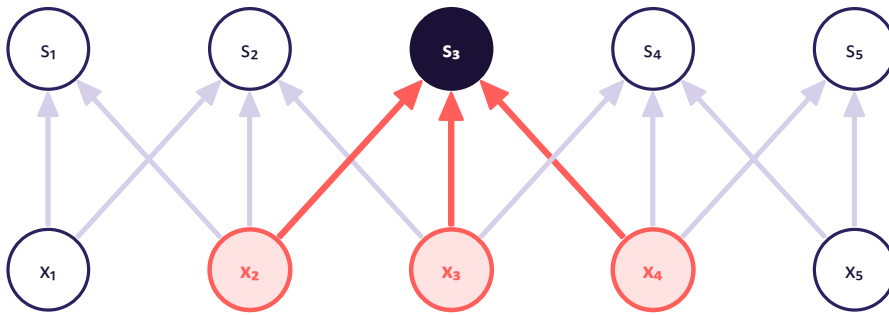
- Sparse interactions — small kernels, few connections
- Parameter sharing — one kernel reused everywhere
- Equivariant representations — shift in, shift out



# Key idea 1 — Sparse interactions

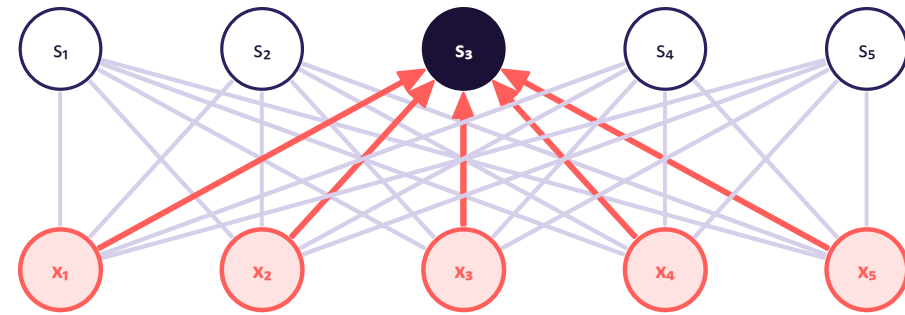
Traditional fully connected layers use a separate parameter for every input–output pair, so every output unit interacts with every input unit. Convolutional layers make the kernel smaller than the input, so each output sees only a local patch.

## Sparse connections — kernel of width 3



Only three inputs —  $x_2, x_3, x_4$  — affect  $s_3$

## Dense connections — fully connected layer



Every one of the five inputs affects  $s_3$

Dense layer

**$m \times n$**

parameters,  $O(m \times n)$  runtime

Convolutional layer

**$k \times n$**

parameters, with  $k \ll m$

- Fewer parameters → lower memory cost and better statistical efficiency
- Fewer operations → the forward pass gets dramatically cheaper
- An image may have millions of pixels, yet edges are detectable with kernels covering only tens of pixels

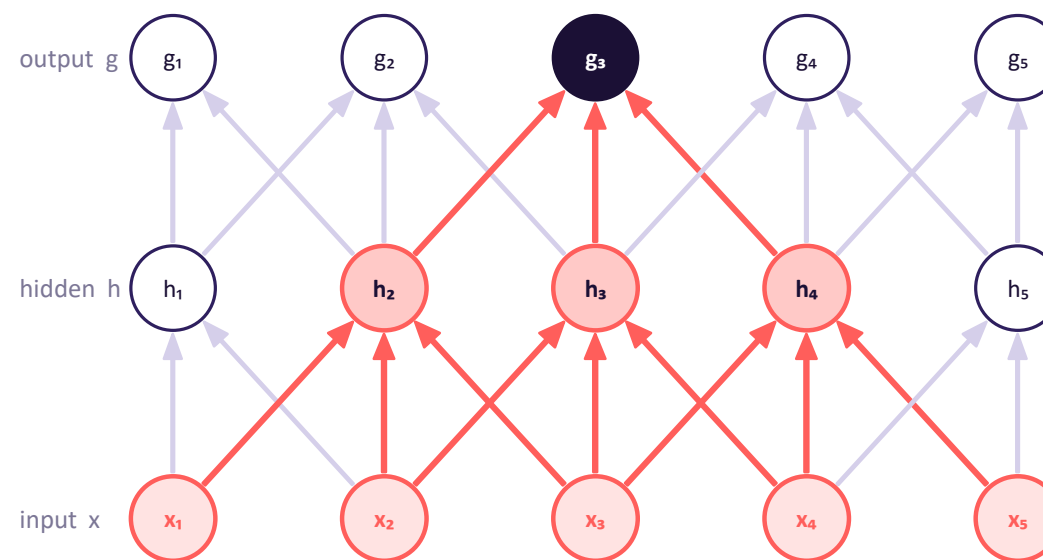


# Sparse now, global later: growing receptive fields

- Units in deeper layers connect only to a small patch of the layer below — but that patch was itself built from a patch of the layer below it.
- So deeper layers indirectly interact with a much larger portion of the input.
- The network describes complicated interactions among many variables using simple building blocks that each describe only sparse interactions.
- This is the same argument VGG later turns into a design rule: stack small filters instead of using one large one.

*Two layers of width-3 kernels already give the top unit a receptive field of five inputs — and it keeps growing with depth.*

Two stacked convolutional layers, kernel width 3



$g_3$  sees only three units of  $h$  — but through them it is influenced by all five inputs.



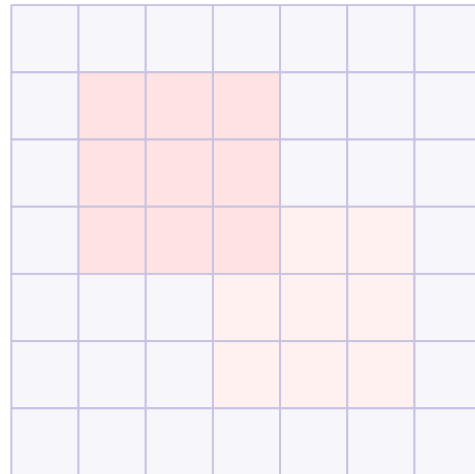
# Key idea 2 — Parameter sharing

Parameter sharing means using the same parameter for more than one function in a model. In a traditional network each weight is used exactly once; in a CNN each member of the kernel is used at every position of the input.

## One kernel, every location

|       |       |       |
|-------|-------|-------|
| $w_1$ | $w_2$ | $w_3$ |
| $w_4$ | $w_5$ | $w_6$ |
| $w_7$ | $w_8$ | $w_9$ |

9 learned weights



applied at every position of the input

## Tied weights

When the weight applied to one input is tied to a weight applied to another input, the network is said to have tied weights. Rather than learning a separate set of parameters for every location, we learn only one set.

- Runtime is unchanged — still  $O(k \times n)$  for the forward pass
- Storage drops to just  $k$  parameters instead of  $m \times n$
- Since  $m$  and  $n$  are usually similar,  $k$  is practically insignificant next to  $m \times n$

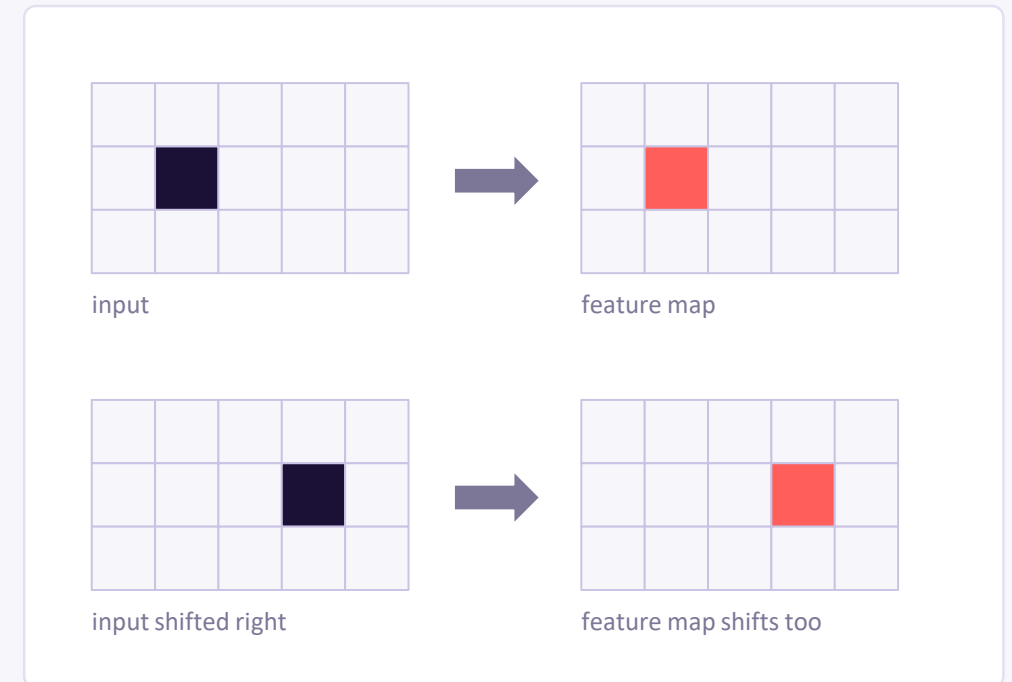


# Key idea 3 — Equivariance to translation

A function  $f$  is equivariant to a function  $g$  if  $f(g(x)) = g(f(x))$

- Shifting the input and then convolving gives the same result as convolving and then shifting the output.
- If we move an object in the image, its representation moves by the same amount in the feature map.
- This is useful whenever a function of a few neighbouring pixels is worth applying everywhere — edge detection is the classic example, since the same edges appear all over an image.

Careful: convolution is NOT naturally equivariant to scaling or rotation. Those need other mechanisms — data augmentation, multi-scale features, or specialised architectures.





# 03

## Anatomy of a convolutional layer

- Output size, stride and zero padding
- Pooling: what it summarises and why
- A fully worked parameter-counting problem



# The parameters of a convolutional layer

- The layer's parameters are a set of learnable filters.
- Each neuron connects only to a local region of the input volume — its receptive field, which is exactly the filter size.
- A filter always spans the full depth of the input volume, plus one bias term.

## Worked example

Input  $32 \times 32 \times 3$  (an RGB CIFAR-10 image), three  $5 \times 5$  filters with stride 1.

How many parameters does one filter have?

$$5 \times 5 \times 3 + 1$$

width  $\times$  height  $\times$  input depth + bias

**= 76 parameters per filter**

three filters  $\rightarrow 76 \times 3 = 228$  parameters in the whole layer

## The rule to memorise

$$\text{params} = (K \times K \times D_{\text{in}} + 1) \times K_{\text{out}}$$

Notice what is absent: the spatial size of the input. A conv layer costs the same whether it sees a  $32 \times 32$  or a  $512 \times 512$  image.



# Output size and stride

$$\text{Output size} = (N - K) / S + 1$$

N = input size   K = kernel size   S = stride

The result must be an integer, otherwise the filter does not fit the input cleanly.

## Try it

$$N = 5 \quad K = 3 \quad S = 1 \quad (5 - 3) / 1 + 1 = 3$$

$$N = 7 \quad K = 3 \quad S = 2 \quad (7 - 3) / 2 + 1 = 3$$

$$N = 32 \quad K = 5 \quad S = 1 \quad (32 - 5) / 1 + 1 = 28$$

$$N = 6 \quad K = 3 \quad S = 2 \quad (6 - 3) / 2 + 1 = 2.5 \quad \times$$

## Stride is a downsampling knob

stride 1 — dense, overlapping windows



stride 2 — the window jumps, output halves



Larger strides mean fewer computations and a smaller output — an alternative to pooling for downsampling.



# Zero padding: keeping the output the same size

- Without padding, the edges of the image are only partially processed and the output is smaller than the input.
- Padding surrounds the input with zeros so the kernel can be centred on border pixels too.
- With stride 1, a padding of  $P = (K - 1) / 2$  gives an output exactly the same size as the input — often called 'same' padding.
- With padding, the size formula becomes  $(N - K + 2P) / S + 1$ .

$$K = 3 \rightarrow P = 1 \quad K = 5 \rightarrow P = 2 \quad K = 7 \rightarrow P = 3$$

5 × 5 input padded with one ring of zeros → 7 × 7

|   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|
| 0 | 0 | 0 | 0 | 0 | 0 | 0 |
| 0 | 0 | 2 | 2 | 0 | 1 | 0 |
| 0 | 2 | 1 | 0 | 1 | 1 | 0 |
| 0 | 2 | 1 | 1 | 0 | 2 | 0 |
| 0 | 0 | 0 | 2 | 2 | 1 | 0 |
| 0 | 1 | 2 | 2 | 0 | 2 | 0 |
| 0 | 0 | 0 | 0 | 0 | 0 | 0 |

The 3 × 3 kernel can now start centred on the true corner pixel, so the output stays 5 × 5.



# The four hyper-parameters of a conv layer

**K****Kernel width**

How big a patch each neuron sees. Wider kernels add parameters and shrink the output, but capture more context.

**K\_out****Kernel depth**

How many filters we use — this becomes the depth of the output volume and controls representational capacity.

**S****Stride**

How far the filter moves between positions, typically 1 or 2. Larger strides downsample the representation.

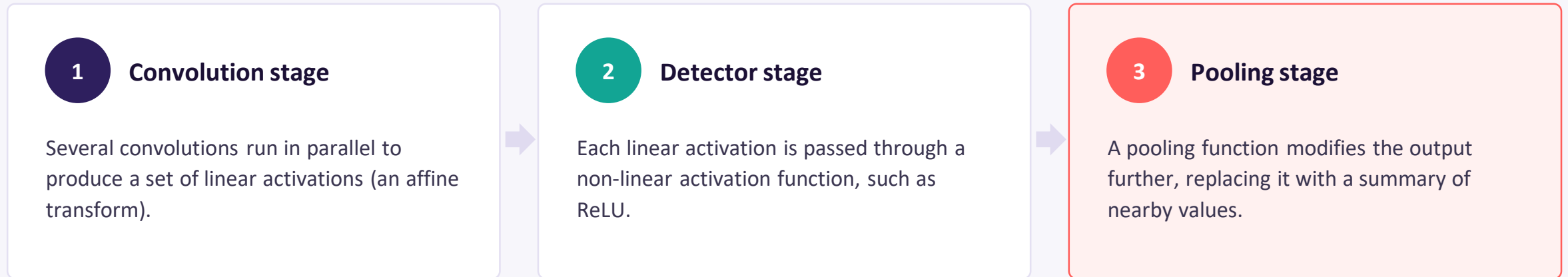
**P****Zero padding**

How many rings of zeros surround the input. Keeps the representation large, at extra time and memory cost.



# Pooling: the third stage of a conv layer

A typical layer of a convolutional network runs in three stages:



## What pooling does

A pooling function replaces the output of the net at a certain location with a summary statistic of the nearby outputs.

## Common pooling functions

**Max pooling** — the maximum output within a rectangular neighbourhood (Zhou & Chellappa, 1988)

**Average pooling** — the mean of a rectangular neighbourhood

**L2-norm pooling** — the L2 norm of a rectangular neighbourhood

**Weighted average** — weighted by distance from the central pixel



# Max pooling in action

Single depth slice (4 × 4)

|   |   |   |   |
|---|---|---|---|
| 1 | 1 | 2 | 4 |
| 5 | 6 | 7 | 8 |
| 3 | 2 | 1 | 0 |
| 1 | 2 | 3 | 4 |

max pool, 2 × 2 filter,  
stride 2

Output (2 × 2)

|   |   |
|---|---|
| 6 | 8 |
| 3 | 4 |

Pooling uses the same size formula

$$(4 - 2) / 2 + 1 = 2$$

Pooling has no learnable parameters at all — it is a fixed function, so it adds zero weights to the model.

- A 2 × 2 max pool with stride 2 discards 75 % of the activations and keeps only the strongest response in each window.
- Pooling is applied independently to every depth slice, so the number of channels never changes.
- Modern architectures increasingly replace pooling with strided convolutions — ResNet uses stride-2 convs, and GoogLeNet ends with global average pooling.

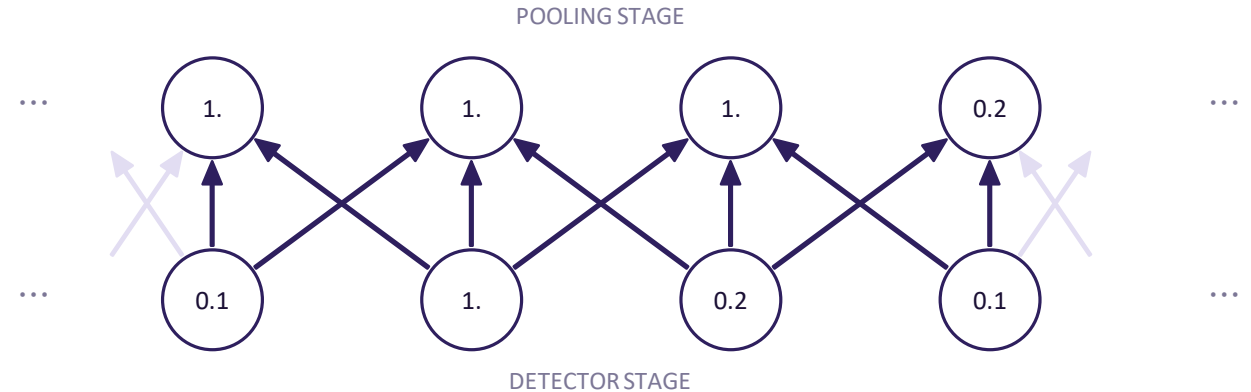


# Why pooling? Invariance to translation

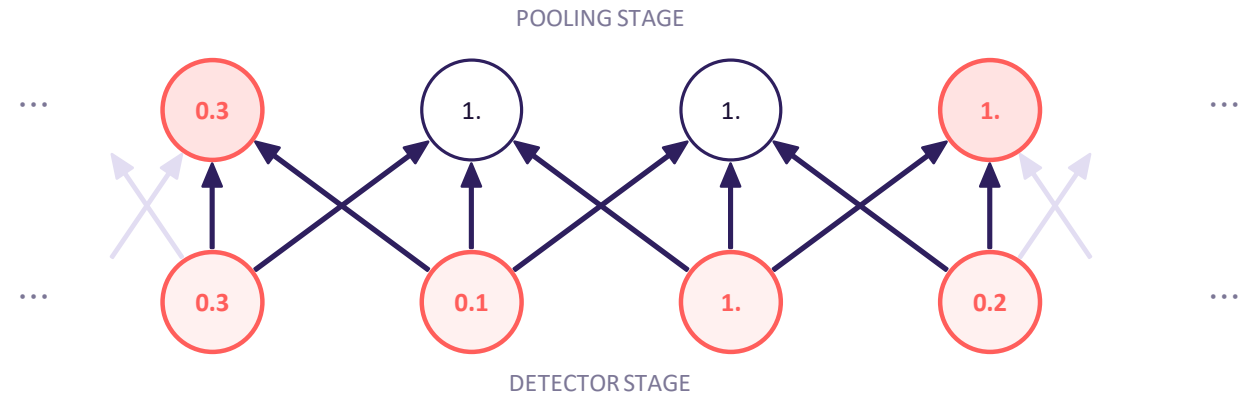
- Pooling helps the representation become approximately invariant to small translations of the input: if we translate the input by a small amount, the values of most of the pooled outputs do not change.
- Invariance to local translation is a very useful property if we care more about whether some feature is present than exactly where it is.
- When determining whether an image contains a face, we need not know the location of the eyes with pixel-perfect accuracy — only that there is an eye on the left side and an eye on the right side.

*Every value in the detector row has changed, but only half of the values in the pooling row have changed.*

Max pooling with a stride of one and a pooling width of three



the same network, after the input has been shifted to the right by one pixel



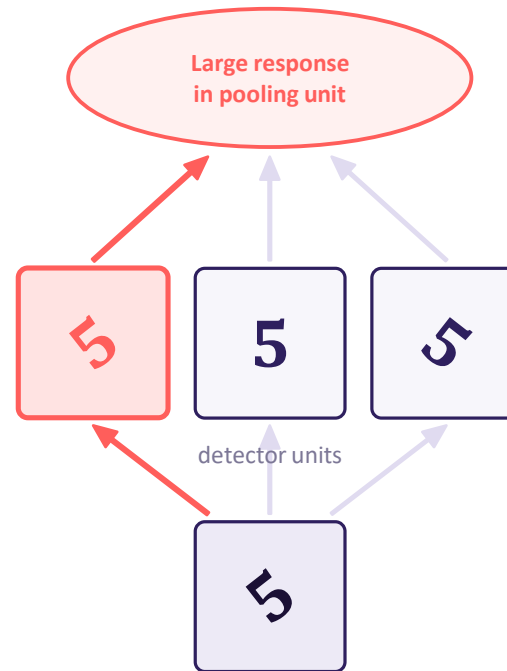


# Why pooling? Learning invariances

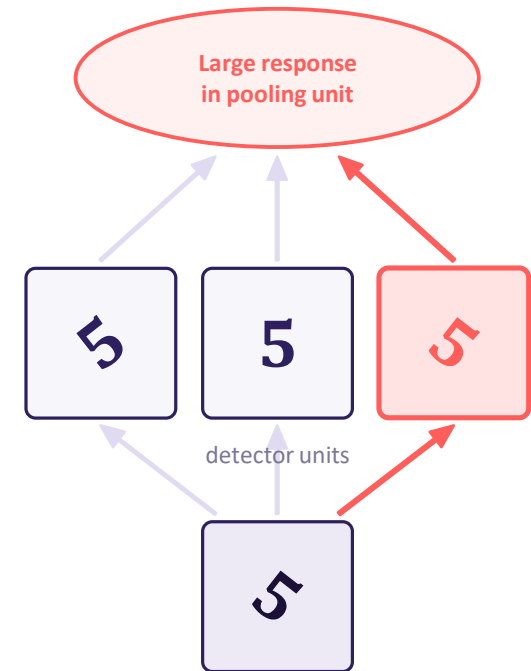
- In this example we use three filters, each tuned to a slightly different orientation of the digit 5.
- When a 5 appears in the input, the corresponding filter matches it and causes a large activation in one detector unit.
- The max-pooling unit then has a large activation regardless of which detector unit was activated.
- If we pool over the outputs of separately parameterized convolutions, the features can learn which transformations to become invariant to.

*Same pooling output, different detector — the invariance is learned, not built in.*

Three filters, each matching a different orientation — one shared pooling unit



input matches detector unit 1

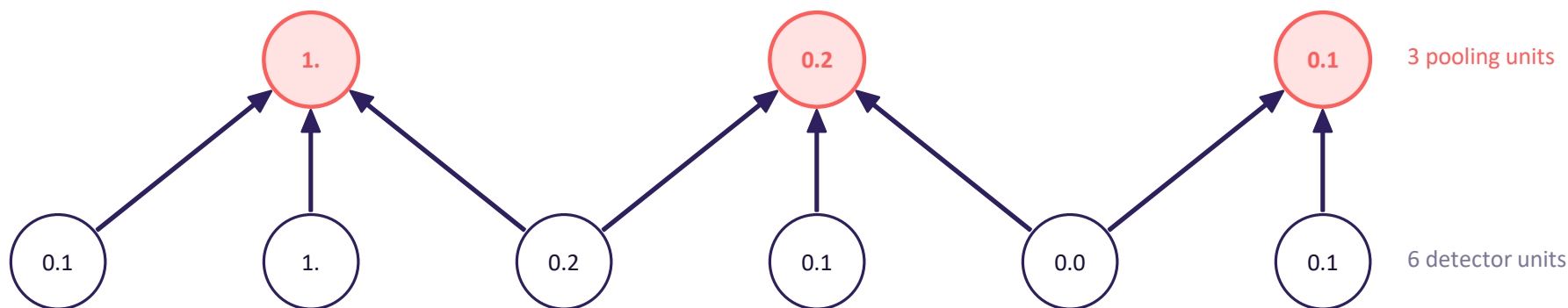


input matches detector unit 3



# Why pooling? Down sampling

Max pooling with a pool width of three and a stride between pools of two



*This reduces the representation size by a factor of two, which reduces the computational and statistical burden on the next layer.*



## Fewer pooling units than detector units

Because pooling summarises the responses over a whole neighbourhood, we can report summary statistics for regions spaced  $k$  pixels apart rather than 1 pixel apart.



## Computational efficiency

The next layer then has roughly  $k$  times fewer inputs to process — here  $k = 2$ , so the representation halves.



## Statistical efficiency and memory

The reduction in input size also improves statistical efficiency and reduces the memory needed to store the parameters.



# Solved problem: counting parameters

Consider a CNN with the following layers:

**Input layer**

one channel,  $32 \times 32$

**Convolutional layer**

6 filters of size  $5 \times 5$ , padding 2, stride 1

**Pooling layer**

$2 \times 2$  max pooling, stride 2, padding 0

**Fully connected layer**

N output neurons

Hint: padding adds 2 rows and 2 columns on each side — 4 in total per dimension.

## Questions

- Compute the dimensions and the number of parameters at both the convolutional and the pooling layers.
- Determine the number of weights in the fully connected layer.



# Solution, step by step



Convolutional layer — parameters

$$6 \times (5 \times 5 + 1 \text{ bias}) = 156$$



Pooling layer — parameters

0 — pooling has nothing to learn



Input size after padding

$$32 + 2 \times 2 = 36$$



Size after the convolution

$$(36 - 5) / 1 + 1 = 32 \quad (\text{same as the input})$$



Convolved feature map

$$32 \times 32 \times 6$$



Size after pooling

$$(32 - 2) / 2 + 1 = 16$$



Input to the fully connected layer

$$16 \times 16 \times 6 = 1\,536 \text{ values}$$

Answer (b)

$$(1\,536 + 1) \times N$$

$$= 1\,537 N \text{ weights}$$

for N output neurons

Take-away: 156 parameters in the conv layer, but over  $1\,500 \times N$  in a single dense layer. The FC layers are where the weights live.



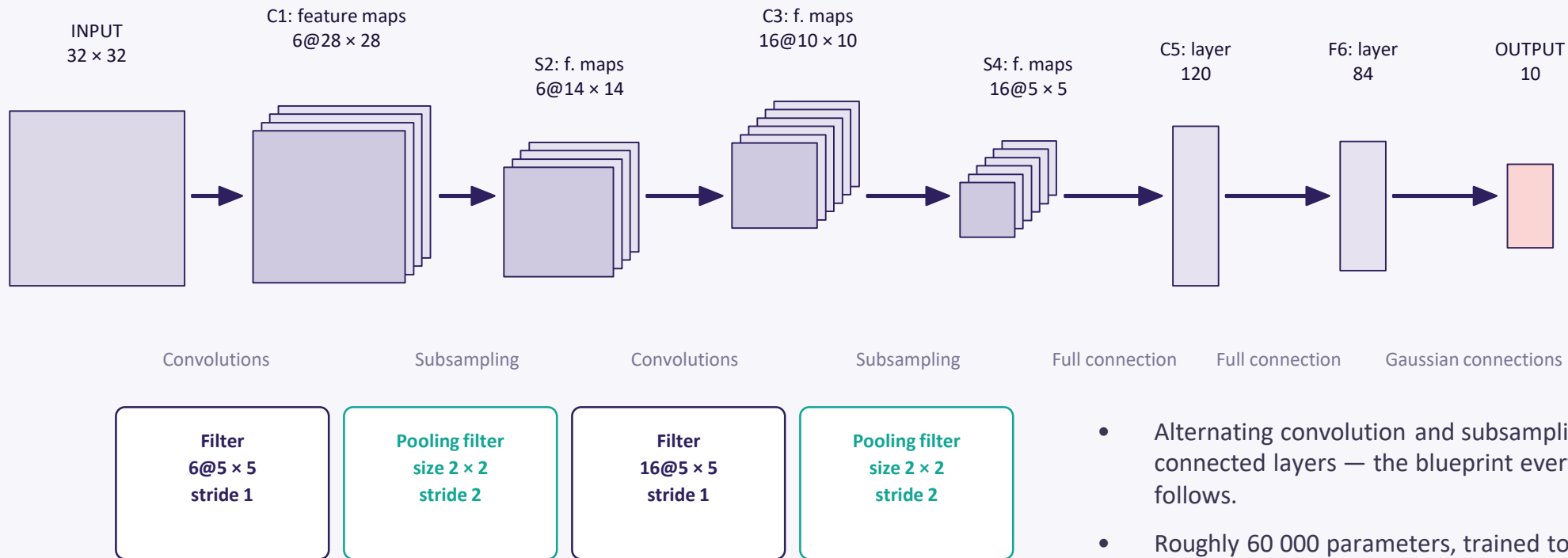
# 04

## Landmark CNN architectures

- LeNet-5 and the ImageNet era: AlexNet, VGG
- GoogLeNet inception module and the  $1 \times 1$  trick
- ResNet, residual learning and transfer learning



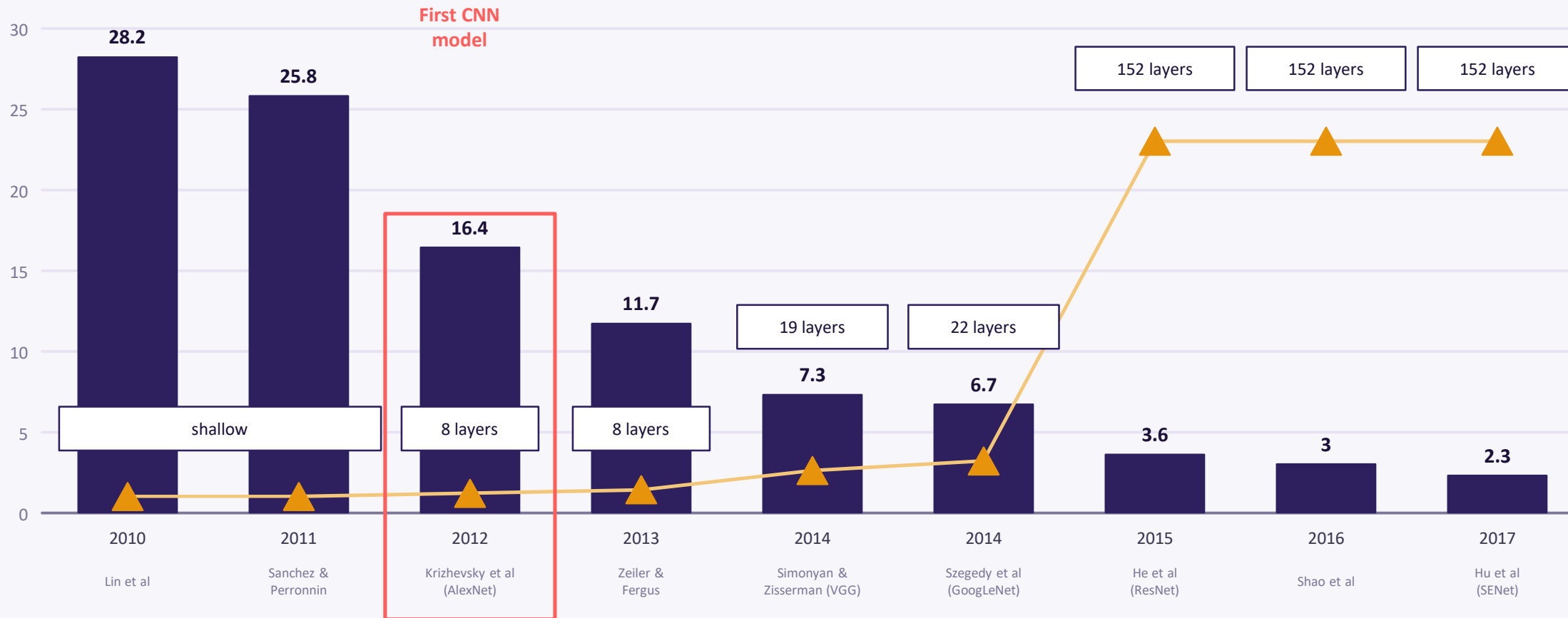
# LeNet-5 · LeCun et al., 1998



- Alternating convolution and subsampling, then fully connected layers — the blueprint every later architecture follows.
- Roughly 60 000 parameters, trained to read handwritten digits on 32 × 32 grayscale images.

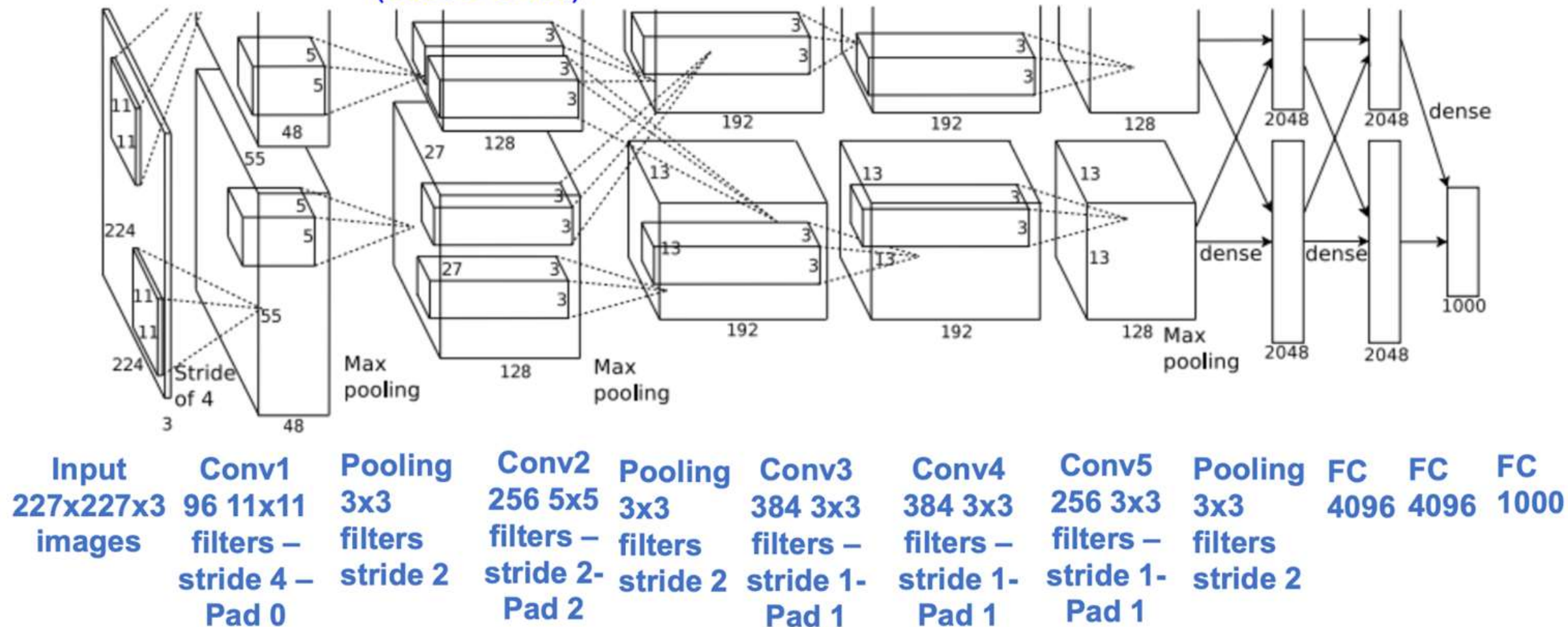


# ImageNet — Large Scale Visual Recognition Challenge (ILSVRC) winners



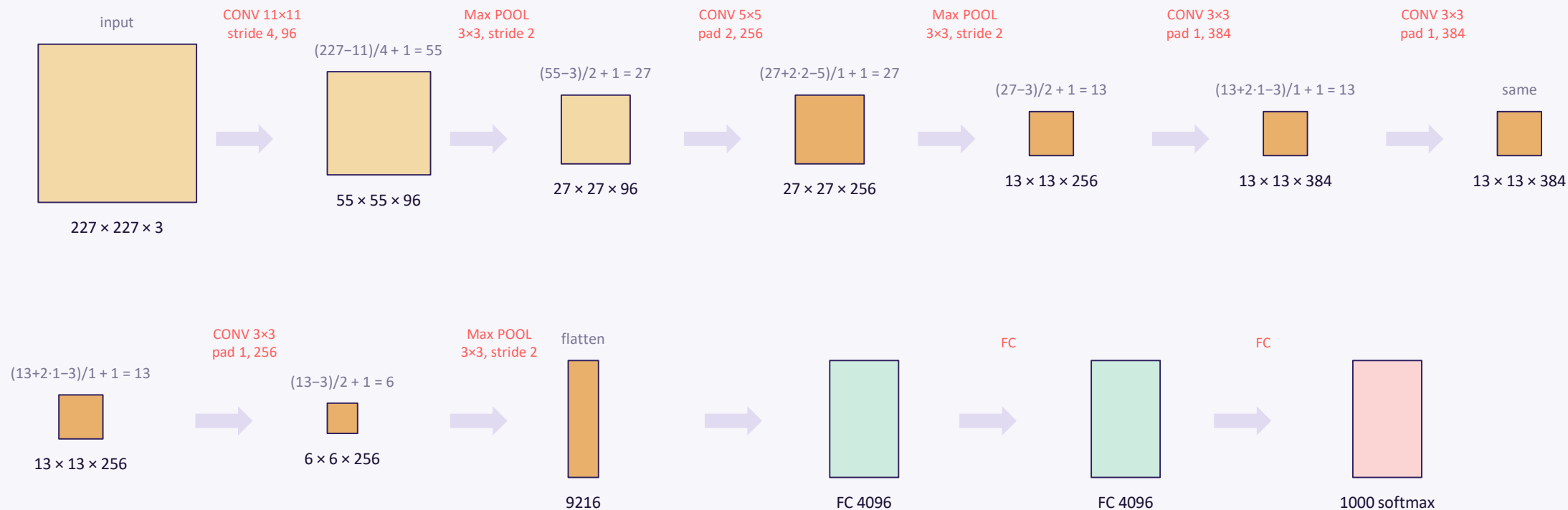
# AlexNet · Krizhevsky et al., 2012

[55x55x48] x 2  
(Used 2 GPUs)





# AlexNet, volume by volume



*The spatial size collapses from 227 to 6 while the depth climbs from 3 to 256 — the standard trade of resolution for semantics.*



# AlexNet — number of parameters

| Layer name  | Tensor size | Weights    | Biases | Parameters | Parameter calc                                                  | Output volume calc                  |
|-------------|-------------|------------|--------|------------|-----------------------------------------------------------------|-------------------------------------|
| Input image | 227×227×3   | 0          | 0      | 0          |                                                                 |                                     |
| Conv-1      | 55×55×96    | 34 848     | 96     | 34 944     | $(11 \times 11 \times 3 + 1) \times 96$                         | $(227 - 11) / 4 + 1 = 55$           |
| MaxPool-1   | 27×27×96    | 0          | 0      | 0          |                                                                 | $(55 - 3) / 2 + 1 = 27$             |
| Conv-2      | 27×27×256   | 614 400    | 256    | 614 656    | $(5 \times 5 \times 96 + 1) \times 256$                         | $(27 + 2 \cdot 2 - 5) / 1 + 1 = 27$ |
| MaxPool-2   | 13×13×256   | 0          | 0      | 0          |                                                                 | $(27 - 3) / 2 + 1 = 13$             |
| Conv-3      | 13×13×384   | 884 736    | 384    | 885 120    | $(3 \times 3 \times 256 + 1) \times 384$                        | $(13 + 2 \cdot 1 - 3) / 1 + 1 = 13$ |
| Conv-4      | 13×13×384   | 1 327 104  | 384    | 1 327 488  | $(3 \times 3 \times 384 + 1) \times 384$                        | $(13 + 2 \cdot 1 - 3) / 1 + 1 = 13$ |
| Conv-5      | 13×13×256   | 884 736    | 256    | 884 992    | $(3 \times 3 \times 384 + 1) \times 256$                        | $(13 + 2 \cdot 1 - 3) / 1 + 1 = 13$ |
| MaxPool-3   | 6×6×256     | 0          | 0      | 0          |                                                                 | $(13 - 3) / 2 + 1 = 6$              |
| FC-1        | 4096×1      | 37 748 736 | 4 096  | 37 752 832 | $(6 \times 6 \times 256 + 1) \times 4096$                       |                                     |
| FC-2        | 4096×1      | 16 777 216 | 4 096  | 16 781 312 | $(4096 + 1) \times 4096$                                        |                                     |
| FC-3        | 1000×1      | 4 096 000  | 1 000  | 4 097 000  | $(4096 + 1) \times 1000$                                        |                                     |
| Output      | 1000×1      | 0          | 0      | 0          |                                                                 |                                     |
| Total       |             |            |        | 62 378 344 | 94 % of all parameters sit in the three fully connected layers. |                                     |

Table: [learnopencv.com](http://learnopencv.com) — number of parameters and tensor sizes in a CNN



# AlexNet — what made it work



## ReLU

One of the first attempts to use rectified linear units in deep learning — several times faster to train than saturating units.



## Local response normalisation

A lateral-inhibition style normalisation across channels. Later architectures dropped it entirely.



## Dropout 0.5

Applied in the fully connected layers, where nearly all the parameters — and the overfitting risk — live.



## SGD momentum

The workhorse optimiser, with a manually decayed learning rate.



## L2 weight decay

Standard regularisation on top of dropout.

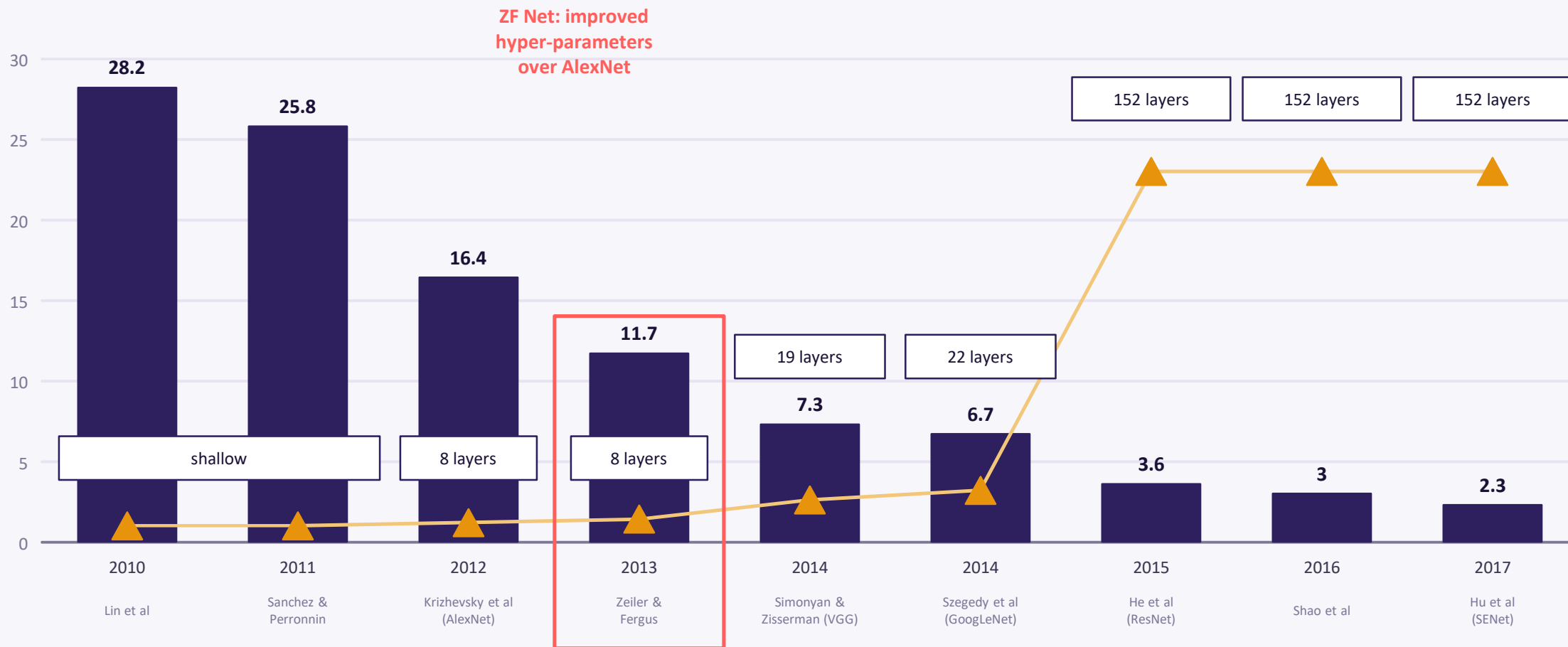


## Heavy data augmentation

Random crops, horizontal flips and PCA colour jitter, effectively enlarging a 1.2 M image dataset many times over.

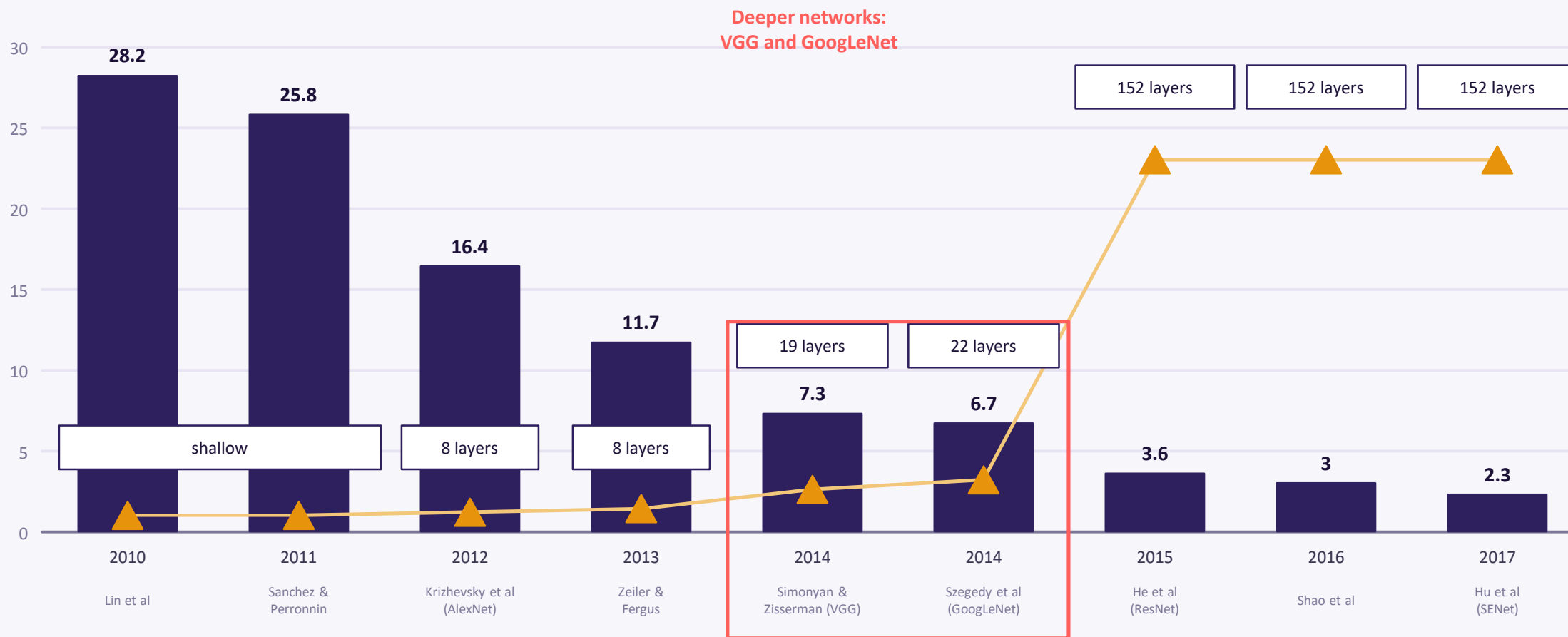


# ILSVRC winners — 2013





# ILSVRC winners — 2014

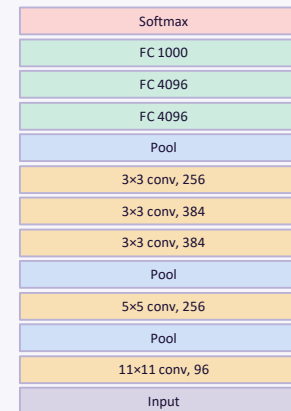




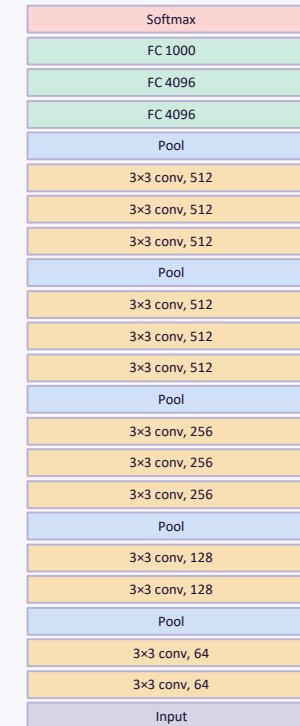
# VGGNet · Simonyan & Zisserman, 2014

Only  $3 \times 3$  conv filters, stride 1, pad 1 · only  $2 \times 2$  max pool, stride 2

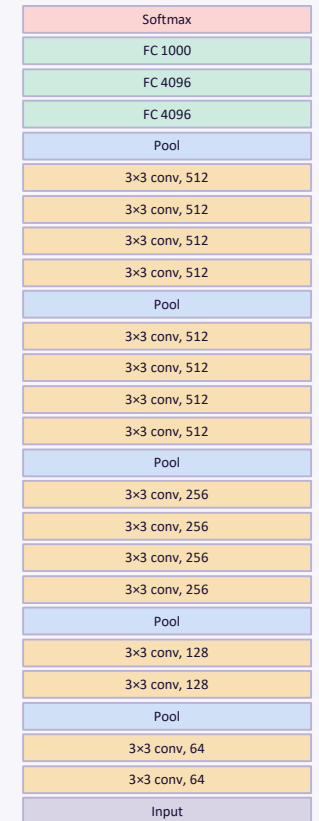
- **Key idea: small filters, deeper networks.**
- Intuition behind using smaller filters: a stack of three  $3 \times 3$  conv layers (stride 1) has the same effective receptive field as one  $7 \times 7$  conv layer.
- But deeper means more non-linearities and fewer parameters:  $3 \cdot (3^2 C^2)$  versus  $7^2 C^2$  for  $C$  channels per layer.
- 16 or 19 weight layers; VGG19 is only slightly better than VGG16 and needs more memory.



AlexNet



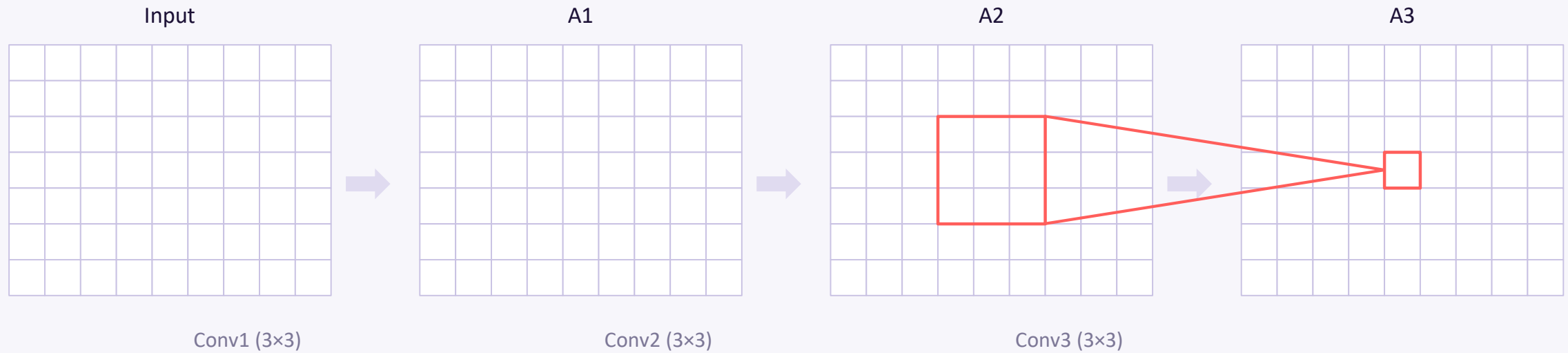
VGG16



VGG19



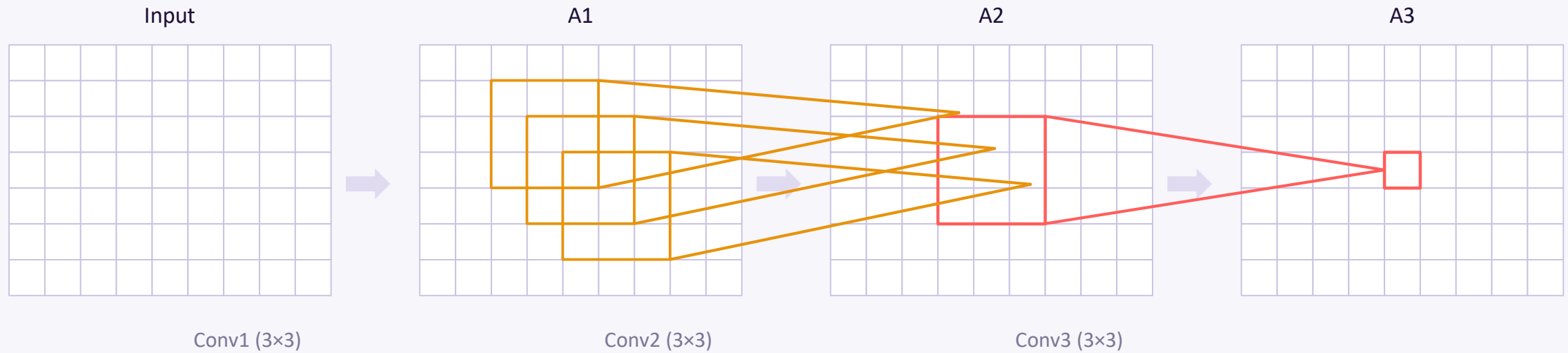
# VGGNet — receptive field (1)



One unit in A3 is computed from a  $3 \times 3$  window of A2. Start at the far right and trace the dependency backwards, one convolution at a time.



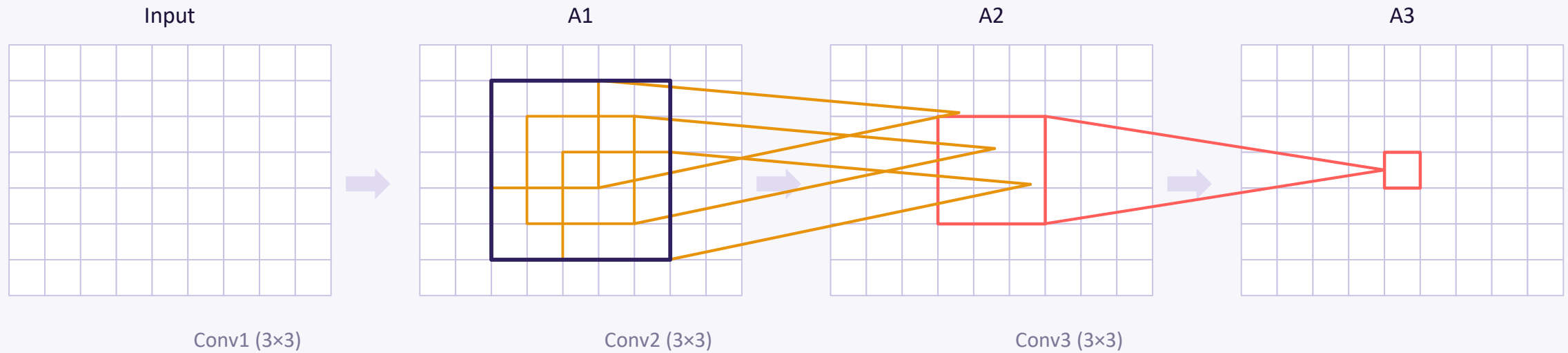
# VGGNet — receptive field (2)



Each of those three A2 units is itself computed from a  $3 \times 3$  window of A1 — and the three windows overlap, sliding one cell apart.



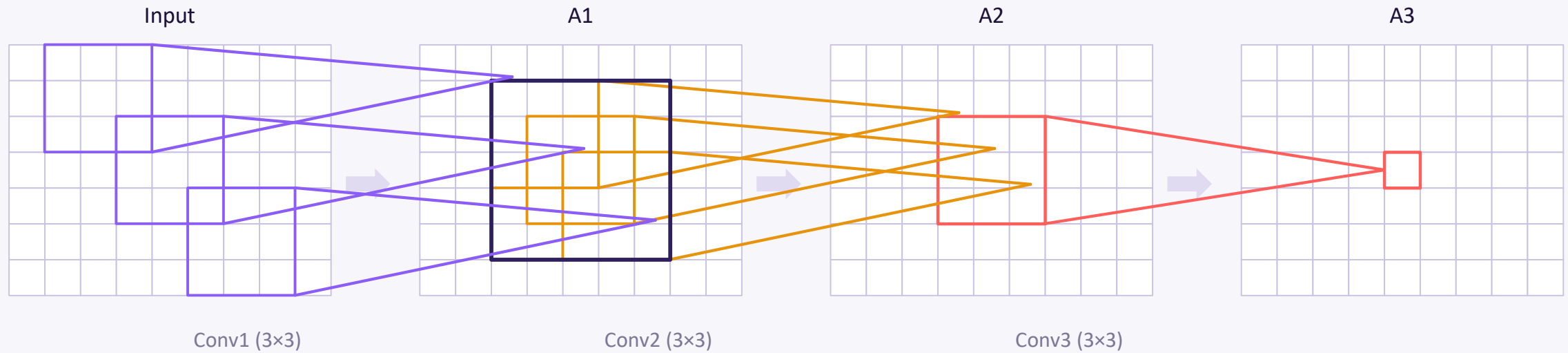
# VGGNet — receptive field (3)



Taken together, the single A3 unit depends on a  $5 \times 5$  region of A1. Two stacked  $3 \times 3$  layers already give a  $5 \times 5$  receptive field.



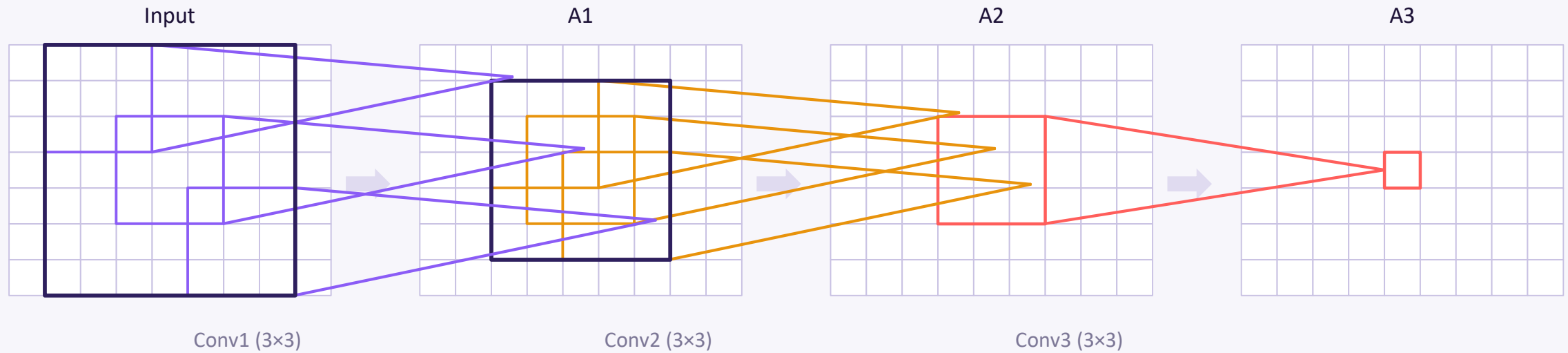
# VGGNet — receptive field (4)



Repeat the argument once more: every unit in that  $5 \times 5$  region of A1 comes from its own  $3 \times 3$  window of the input.



# VGGNet — receptive field (5)



Three stacked  $3 \times 3$  conv layers see a  $7 \times 7$  region of the input — the same receptive field as a single  $7 \times 7$  conv, with  $27C^2$  parameters instead of  $49C^2$  and three ReLUs instead of one.

(not counting biases)

Most memory sits in the early conv layers

|               |
|---------------|
| Softmax       |
| FC 1000       |
| FC 4096       |
| FC 4096       |
| Pool          |
| 3x3 conv, 512 |
| 3x3 conv, 512 |
| 3x3 conv, 512 |
| Pool          |
| 3x3 conv, 512 |
| 3x3 conv, 512 |
| 3x3 conv, 512 |
| Pool          |
| 3x3 conv, 256 |
| 3x3 conv, 256 |
| 3x3 conv, 256 |
| Pool          |
| 3x3 conv, 128 |
| 3x3 conv, 128 |
| Pool          |
| 3x3 conv, 64  |
| 3x3 conv, 64  |
| Input         |

**Most parameters sit in the late FC layers**

## VGG16



# VGG — practical notes



## Similar training procedure to Krizhevsky 2012

SGD with momentum, weight decay, dropout in the FC layers, heavy augmentation.



## No local response normalisation

LRN was dropped: it added computation and memory without improving accuracy.



## VGG16 or VGG19

VGG19 is only slightly better and costs more memory — VGG16 is the usual choice.



## FC7 features generalise well

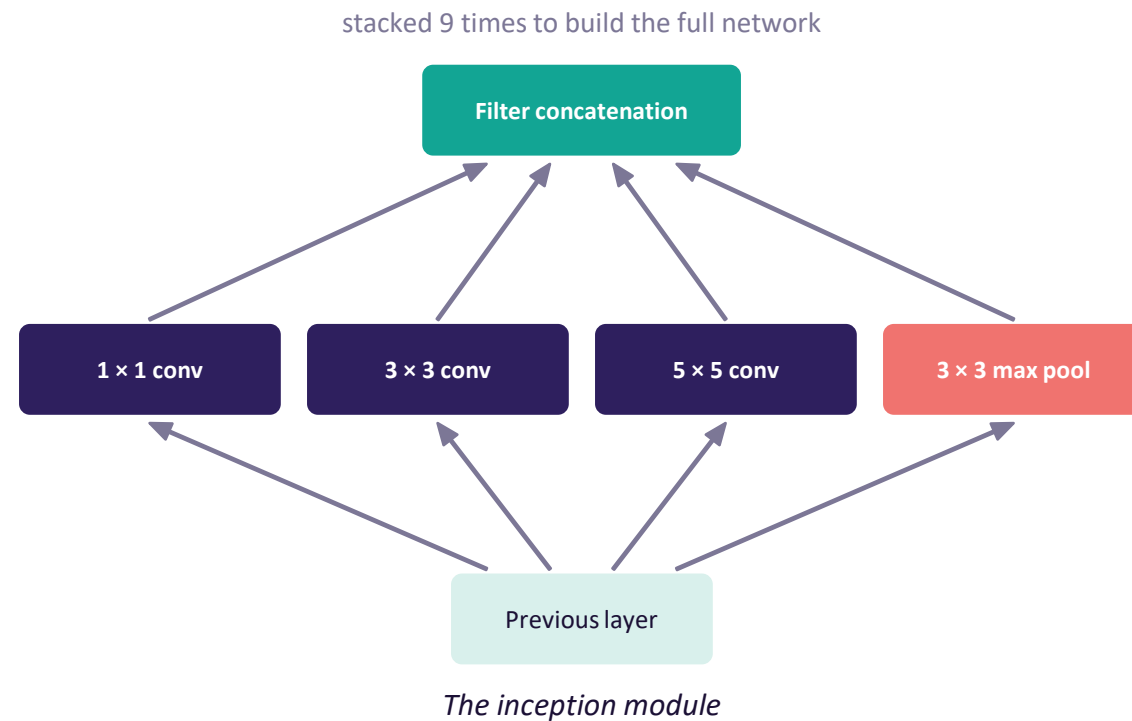
The penultimate layer transfers to other tasks remarkably well, which made VGG the default feature extractor for years.



# GoogLeNet · Szegedy et al., 2014

- Deeper network with computational efficiency
- 22 layers
- Only 5 M parameters
- 12 times less than AlexNet
- 27 times less than VGG-16
- Avoids expensive fully connected layers

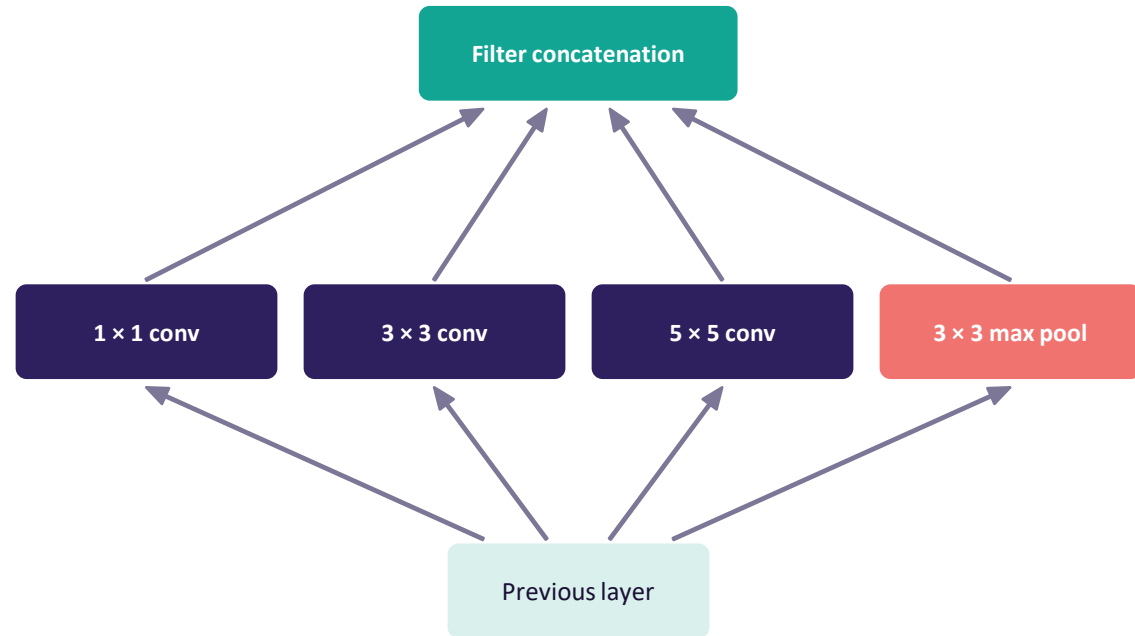
*Basic idea: design a good local network topology — a network within a network — and stack these modules on top of each other.*





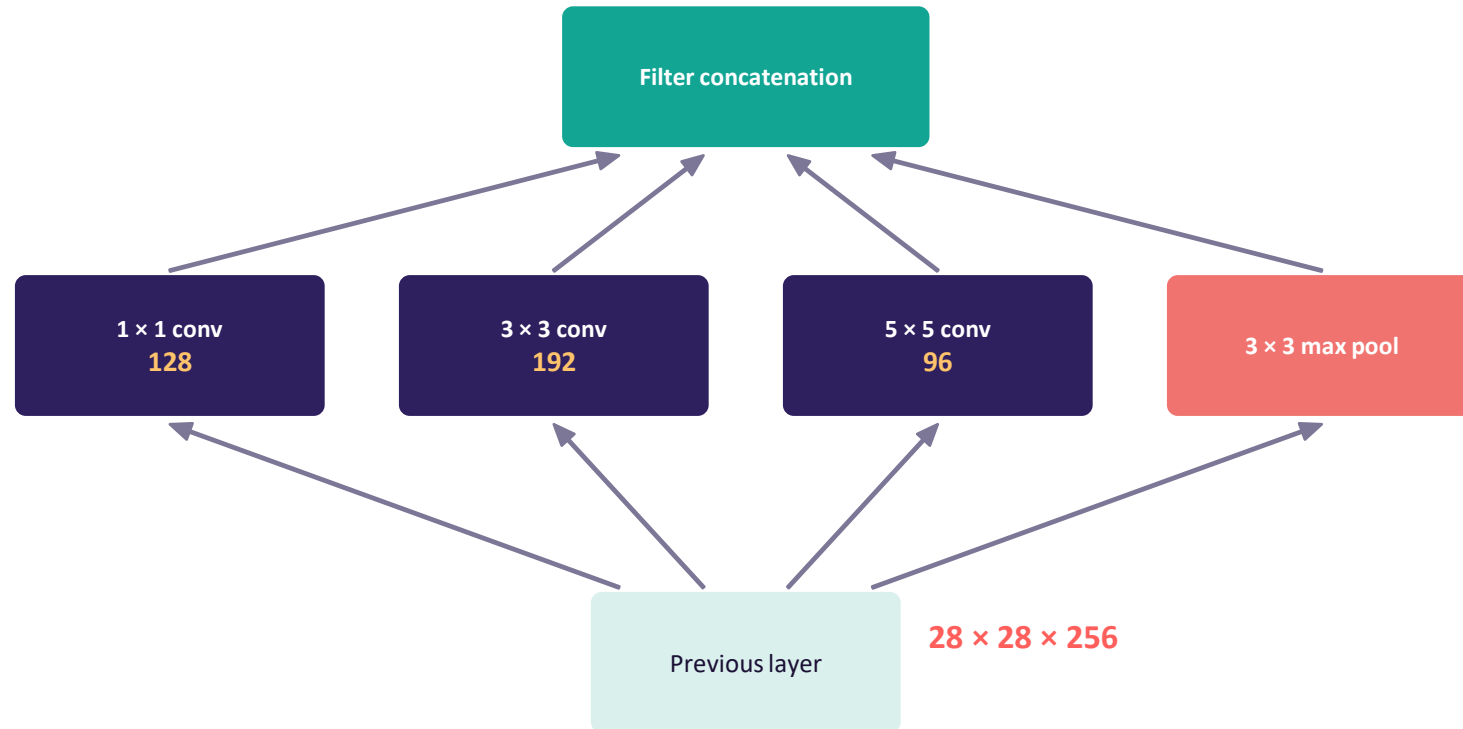
# GoogLeNet — the naive inception module

- Apply parallel filter operations on the input from the previous layer:
  - multiple receptive field sizes for convolution ( $1 \times 1$ ,  $3 \times 3$ ,  $5 \times 5$ )
  - a pooling operation ( $3 \times 3$ )
- Concatenate all filter outputs together channel-wise.
- Let the network choose which receptive field size is useful at each depth, instead of committing to one.





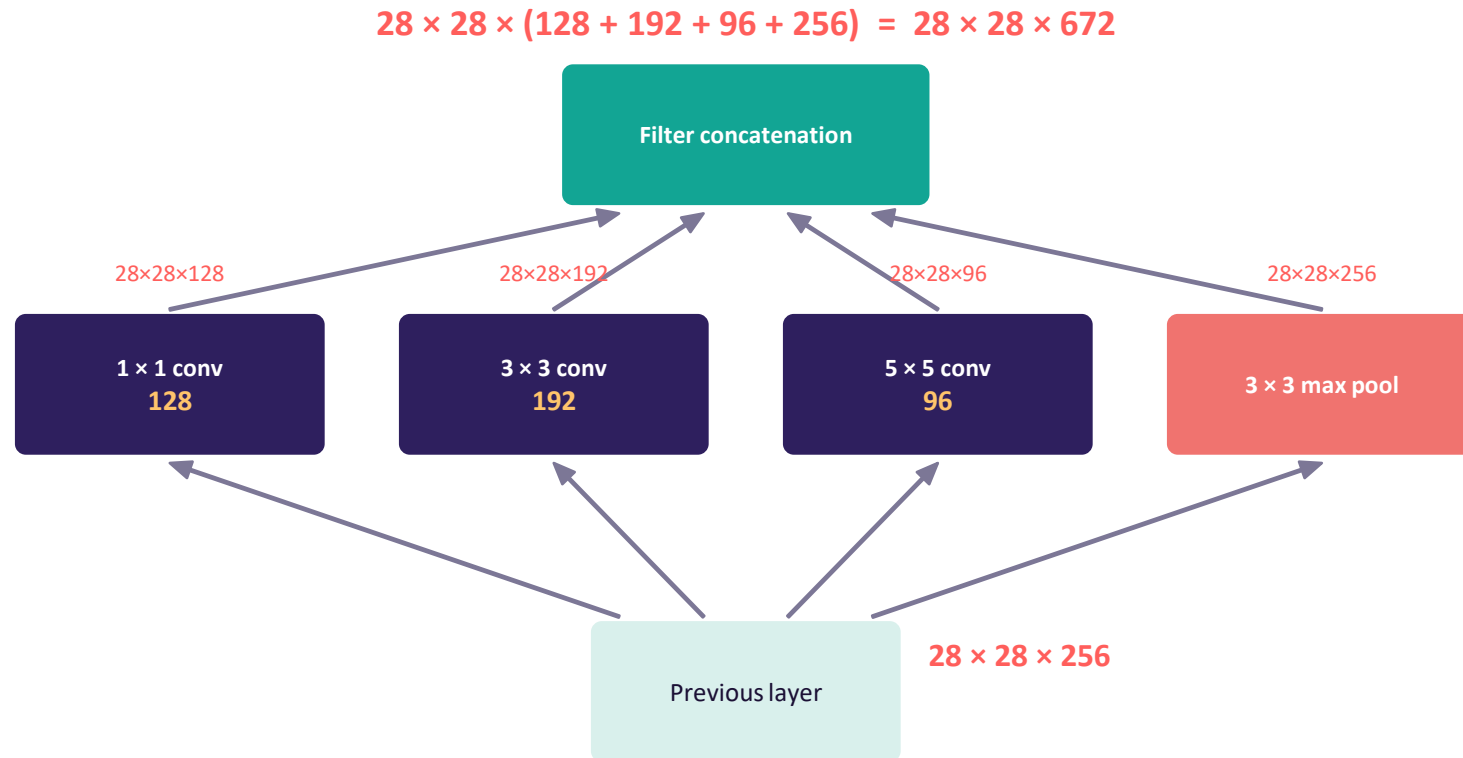
## Example — what is the output size in this case?



*Each convolution uses padding so the spatial size is preserved. Work out the output of every branch, then the concatenated result.*



# Example — what is the problem here?



The pooling branch preserves the input depth, so the concatenated depth is always larger than the input depth.



# Problems with the naive inception module

## Computationally expensive — conv ops

[1 × 1 conv, 128]       $28 \times 28 \times 128 \times 1 \times 1 \times 256$

[3 × 3 conv, 192]       $28 \times 28 \times 192 \times 3 \times 3 \times 256$

[5 × 5 conv, 96]       $28 \times 28 \times 96 \times 5 \times 5 \times 256$

**Total: 854 M ops**

## Depth can only grow

The pooling layer also preserves feature depth, which means the total depth after concatenation can only grow at every layer.

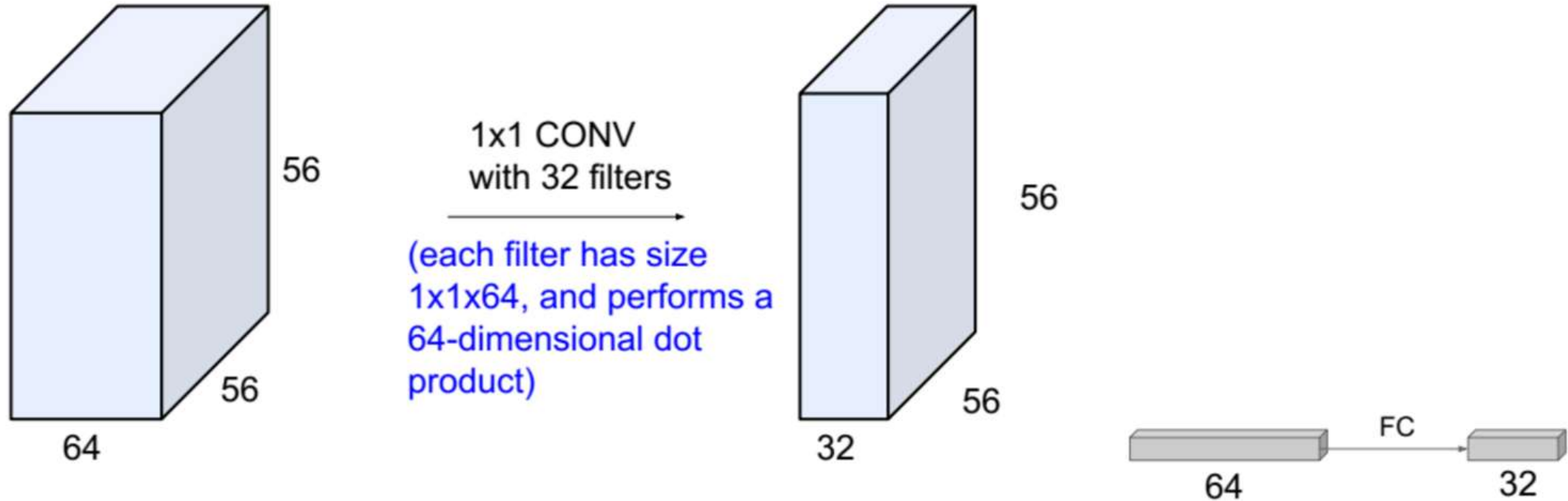
**How can we solve this problem?**

- Nearly a billion operations inside a single module — and GoogLeNet stacks nine of them.
- We need a way to cut the channel dimension without throwing away spatial resolution.



# GoogLeNet — solution: $1 \times 1$ convolutions

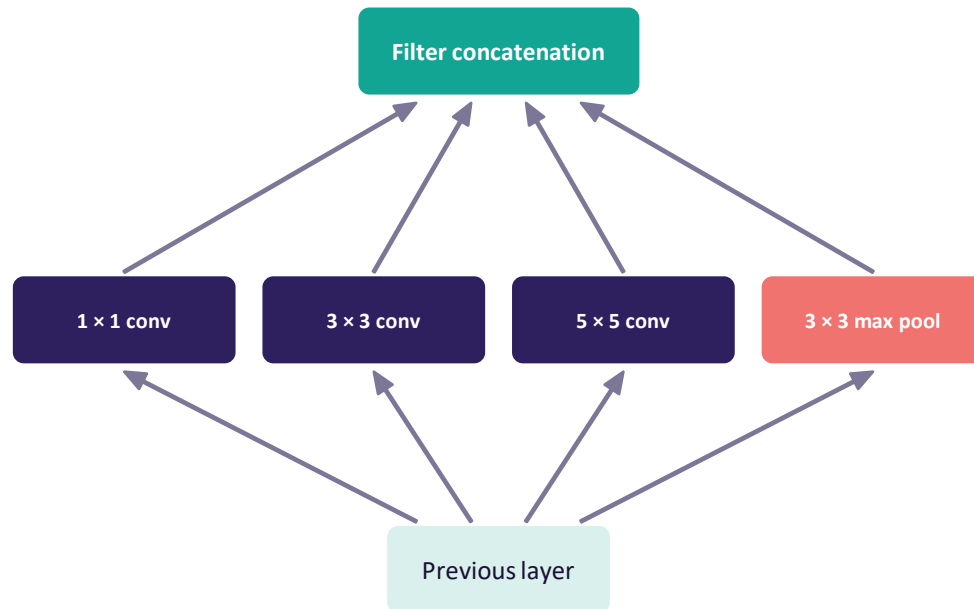
- Use  $1 \times 1$  convolutions to reduce the feature channel size.
- Projects depth to a lower dimension (a combination of feature maps) while preserving the spatial dimensions.



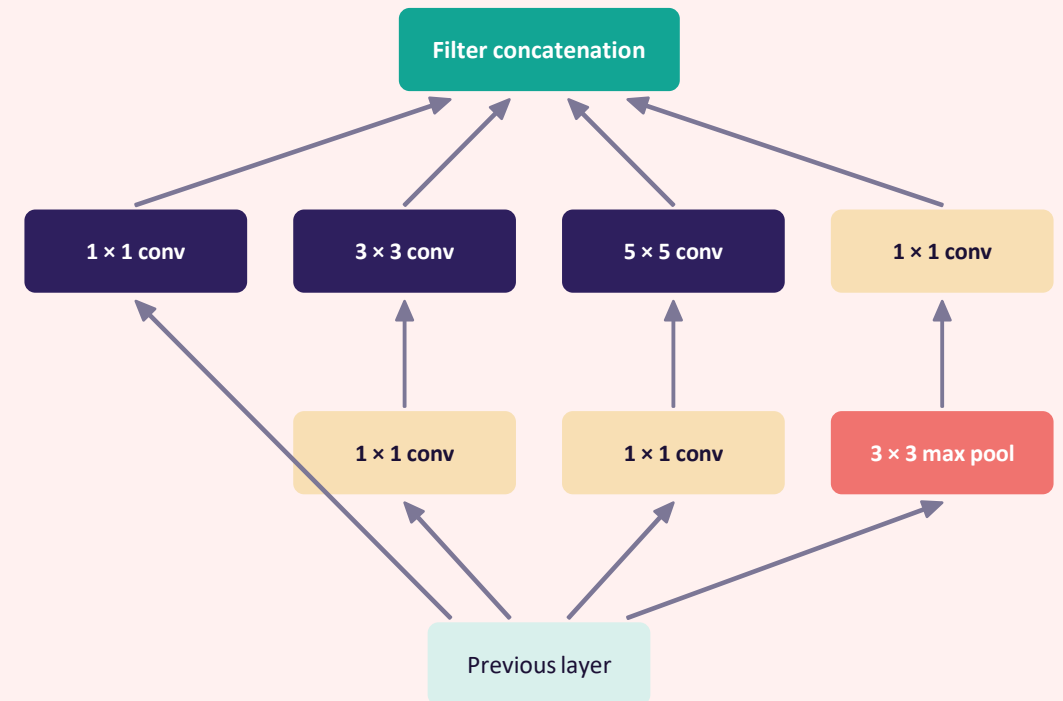


# Inception module with dimensionality reduction

Naive inception module



With dimensionality reduction

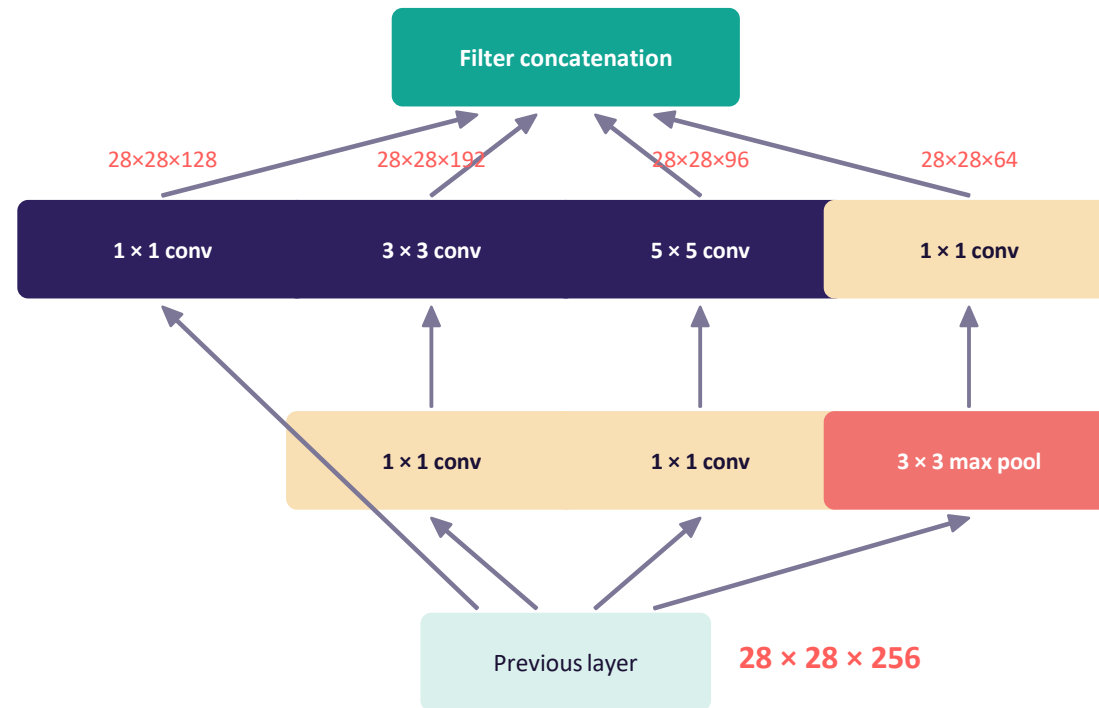


$1 \times 1$  bottlenecks go before the expensive  $3 \times 3$  and  $5 \times 5$  branches, and after the pooling branch.



# Back to the example

$$28 \times 28 \times (128 + 192 + 96 + 64) = 28 \times 28 \times 480$$



672 channels before, 480 after — and the module is now cheaper to compute too.



# The same module, now 358 M ops

## Conv ops

[1 × 1 conv, 64]       $28 \times 28 \times 64 \times 1 \times 1 \times 256$

[1 × 1 conv, 64]       $28 \times 28 \times 64 \times 1 \times 1 \times 256$

[1 × 1 conv, 128]       $28 \times 28 \times 128 \times 1 \times 1 \times 256$

[3 × 3 conv, 192]       $28 \times 28 \times 192 \times 3 \times 3 \times 64$

[5 × 5 conv, 96]       $28 \times 28 \times 96 \times 5 \times 5 \times 64$

[1 × 1 conv, 64]       $28 \times 28 \times 64 \times 1 \times 1 \times 256$

## 358 M ops

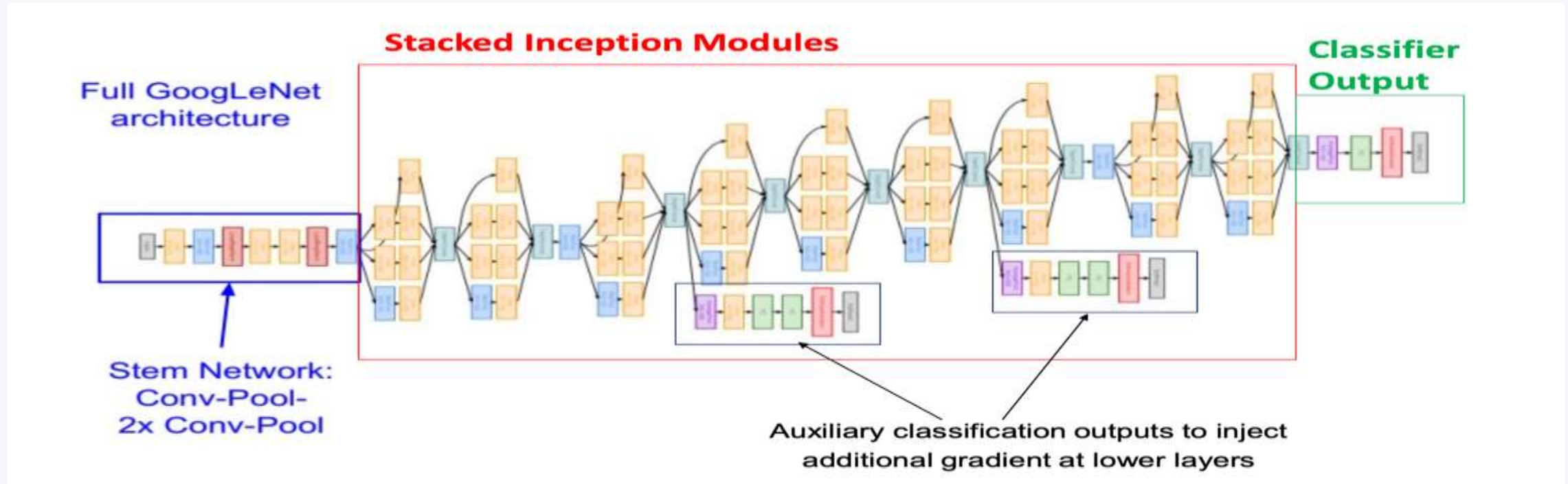
compare to 854 M ops for the naive module — a 58 % reduction

### Where the saving comes from

The 3 × 3 and 5 × 5 convolutions now run over 64 channels instead of 256 — four times less work in exactly the branches that cost the most.



# The full GoogLeNet

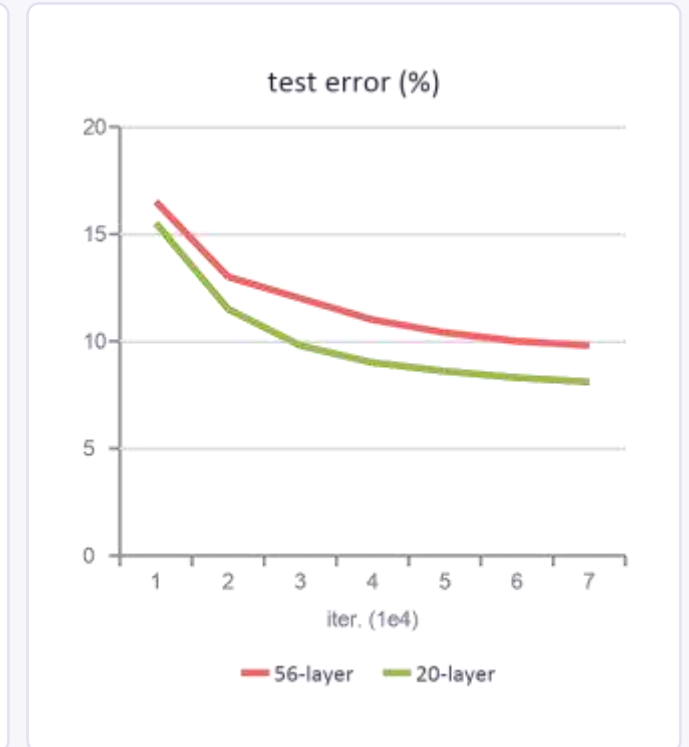
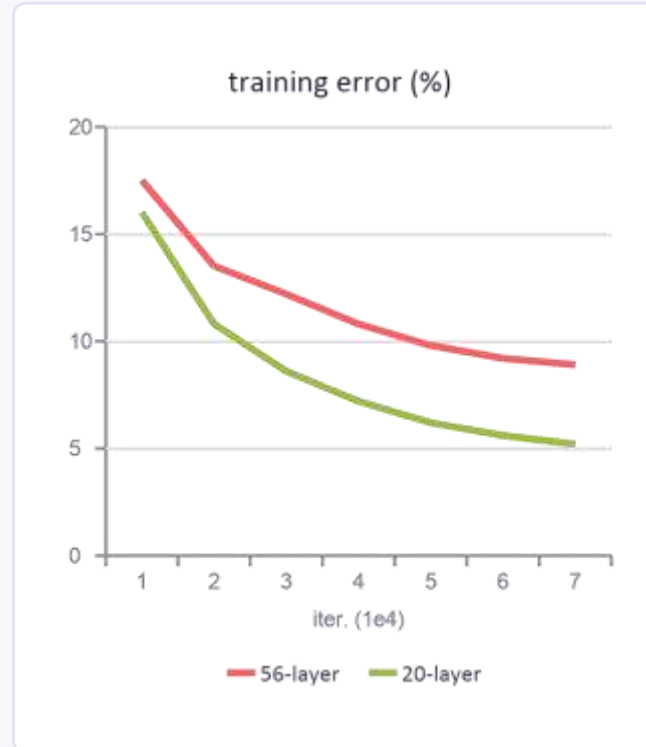


- Stem network, then nine stacked inception modules, then a classifier head — 22 weight layers in total.
- Two auxiliary softmax heads branch off the middle of the network during training and are discarded at inference time.
- No expensive fully connected stack: global average pooling collapses the final feature map before a single FC 1000 layer.



# ResNet · He et al., 2015 — intuition

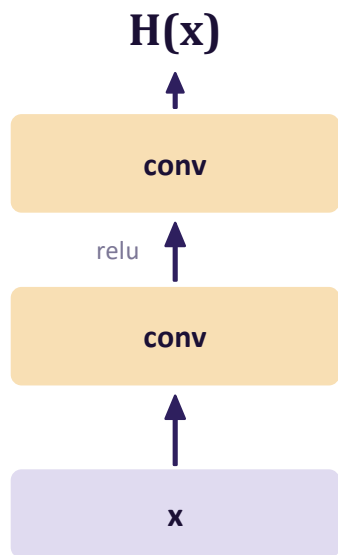
- Is learning better networks as easy as stacking more layers?
- What happens if you keep stacking layers on a “plain” network?
- The deeper network has higher training error, and thus higher test error.
- Fact: deep models have more representation power (more parameters) than shallower models.
- Hypothesis: the problem is an optimisation problem — deeper models are harder to optimise.



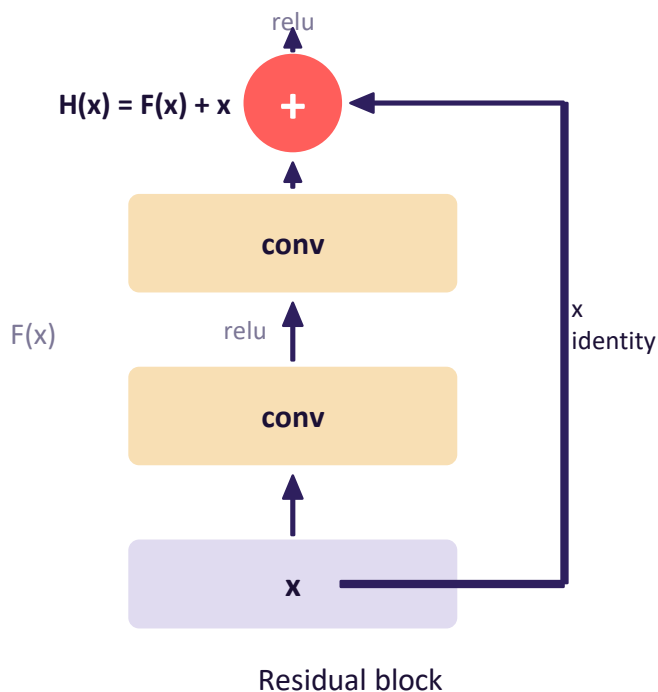


# ResNet — intuition (cont.)

Basic idea: what should the deeper model learn to be at least as good as the shallower model?



“Plain” layers



Residual block

## Identity mapping

$$H(x) = x \text{ if } F(x) = 0$$

Doing nothing becomes the easiest thing for the block to learn, so extra depth can never hurt.

## Idea

$$F(x) = H(x) - x$$

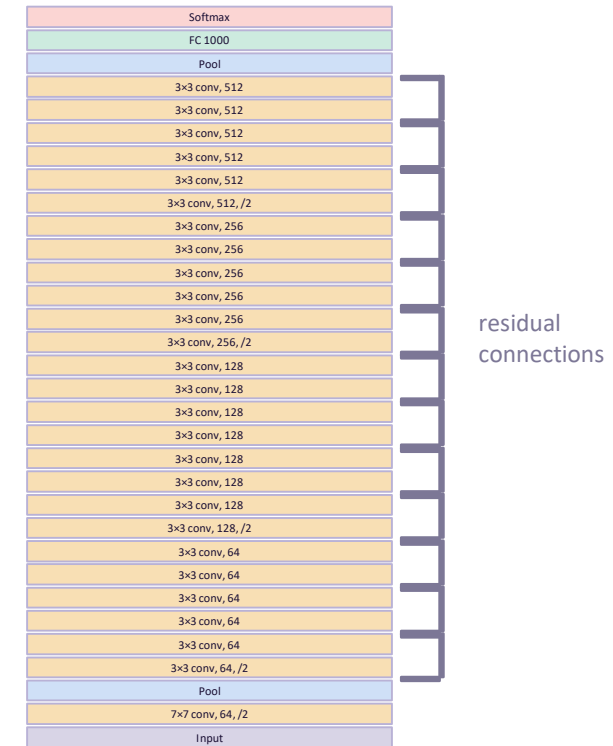
Use the layers to fit the residual  $F(x)$  instead of fitting the desired mapping  $H(x)$  directly.



# The full ResNet architecture

- **Stack residual blocks**
- Every residual block has two  $3 \times 3$  conv layers
- Periodically double the number of filters and downsample spatially using stride 2 (divide by 2 in each dimension)
- Additional conv layer at the beginning (stem)
- No FC layers at the end — only FC 1000 to output the classes
- Total depths of 18, 34, 50, 101 and 152 layers for ImageNet

Batch normalisation after every conv layer · Xavier/He initialisation · no dropout used

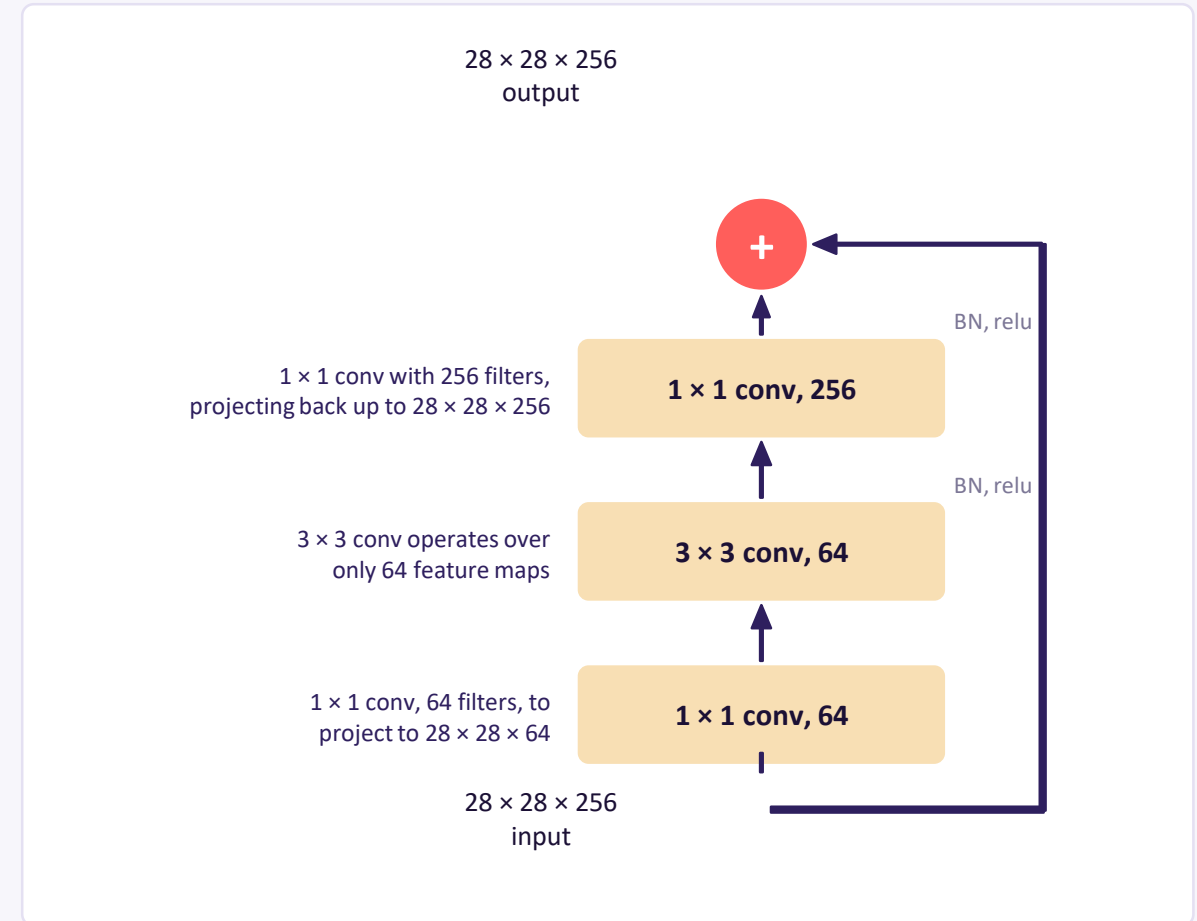


## ResNet-34



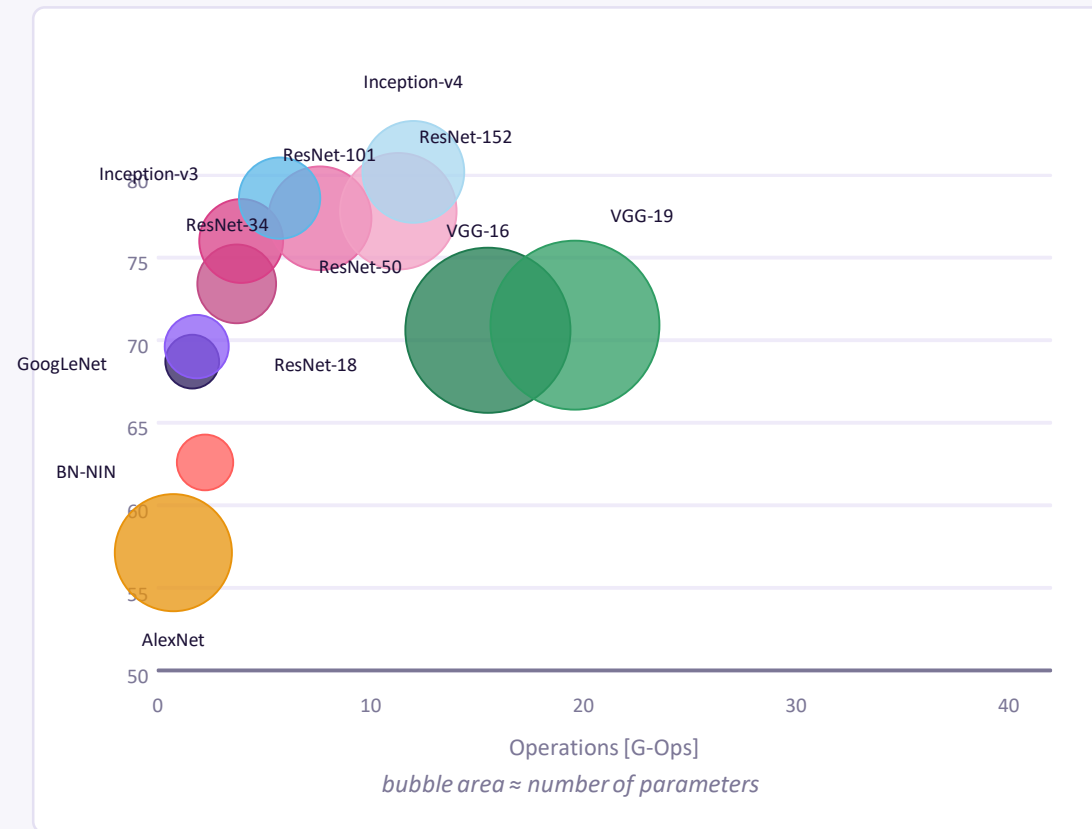
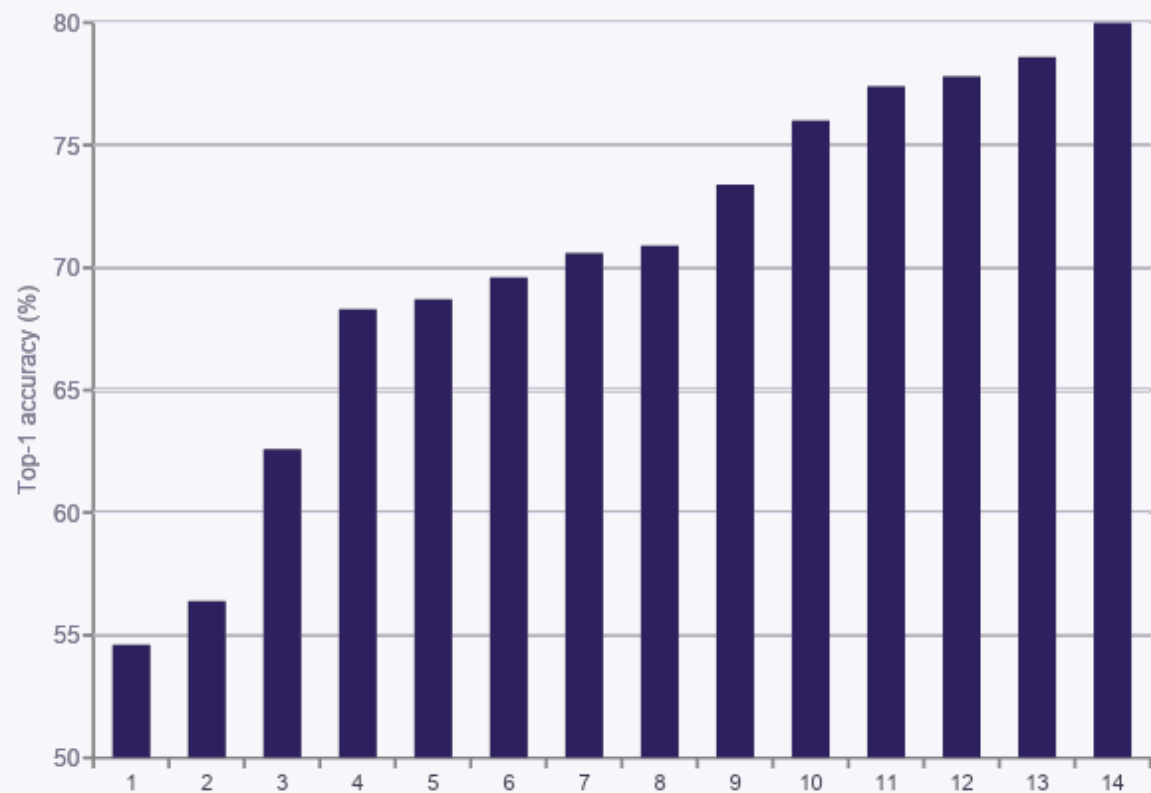
# ResNet — the bottleneck block

- Total depths of 18, 34, 50, 101 and 152 layers for ImageNet.
- For deeper networks (ResNet-50+), use a “bottleneck” layer to improve efficiency — the same trick as GoogLeNet.
- A  $1 \times 1$  conv projects 256 channels down to 64, the  $3 \times 3$  conv does its work cheaply, and a second  $1 \times 1$  conv projects back up.
- Batch normalisation and a ReLU follow each of the first two convolutions.





# Comparing the architectures



Paper: An Analysis of Deep Neural Network Models for Practical Applications, 2017 (Canziani et al.)



# Transfer learning: you rarely train from scratch

Take a network trained on a large dataset such as ImageNet, then reuse its learned representation on your own — usually much smaller — dataset. Early layers learn generic edges and textures that transfer almost everywhere.



## Small dataset, similar domain

Freeze the whole convolutional base and train only a new classifier head. The pretrained features are already right.



## Small dataset, different domain

Freeze the early layers, retrain the later blocks. Low-level features transfer, high-level ones do not.



## Large dataset, similar domain

Fine-tune the whole network with a small learning rate, using the pretrained weights as a very good initialisation.



## Large dataset, different domain

Fine-tune everything, or consider training from scratch if the domain is genuinely far away.

### Practical recipe

Replace the final classifier · freeze the base · train the head · unfreeze the top blocks · fine-tune at a 10× smaller learning rate.

### Course project

Find a dataset of interest (under ~10k images) · check the TensorFlow model garden ([github.com/tensorflow/models](https://github.com/tensorflow/models)) for a pretrained model that learns a similar representation · transfer learn to your dataset.



# 05

## Beyond classification

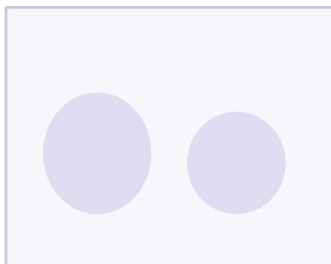
- Object detection: what is where, with boxes
- Semantic segmentation: a class for every pixel
- Instance segmentation: pixels, grouped per object



# The core computer-vision tasks

Every task below runs on the same convolutional backbone. What changes is the output head — and therefore the loss.

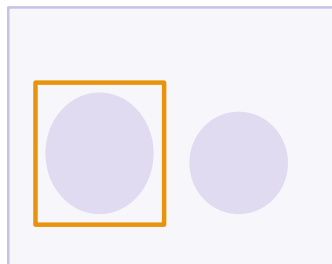
## Classification



CAT

One label for the whole image

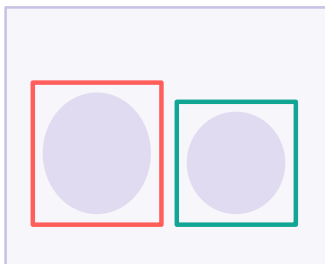
## Classification + localisation



CAT

One label and one bounding box

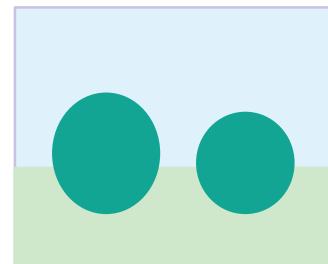
## Object detection



CAT, DOG, DOG

Many objects, each with a box and a label

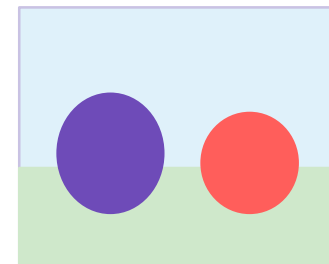
## Semantic segmentation



CAT, GRASS, SKY

A class for every pixel — no object identity

## Instance segmentation



CAT, DOG 1, DOG 2

Per-pixel labels, separated per object



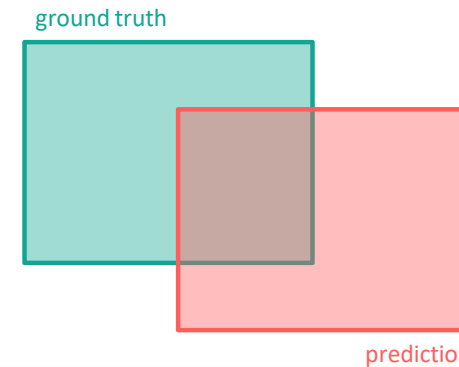
# Object detection: what makes it hard

- A variable number of outputs. A classifier always emits one vector; a detector must emit anywhere from zero to hundreds of boxes.
- Two losses at once: classify each object and regress four box coordinates.
- Huge scale variation — the same class can occupy 20 pixels or 2 000.
- Extreme class imbalance: almost every candidate region is background.
- Many applications need real-time inference on video.

## Two ingredients almost every detector uses

Anchor boxes — a fixed set of reference shapes at every location. Non-maximum suppression (NMS) — collapse overlapping duplicate predictions.

## Intersection over Union (IoU)



$\text{IoU} = \text{area of overlap} / \text{area of union}$

A prediction usually counts as correct at  $\text{IoU} \geq 0.5$ .

## The reporting metric: mAP

Average precision (AP) is the area under the precision–recall curve for one class. Mean AP averages it over all classes.

COCO reports  $\text{mAP}@[.5 : .95]$  — averaged over ten IoU thresholds — which is much stricter than the older  $\text{mAP}@0.5$ .



# Two-stage detectors: the R-CNN family

Stage 1 proposes regions that might contain an object. Stage 2 classifies each proposal and refines its box. Accurate, historically slower.

## R-CNN

2014

Selective search proposes ~2 000 regions; each is warped and pushed through a CNN separately, then an SVM classifies it.

---

*Painfully slow — about 47 s per image at test time, and training is a multi-stage pipeline.*

## Fast R-CNN

2015

Run the CNN once over the whole image, then crop features per proposal with RoI pooling. One network, one multi-task loss.

---

*~25 × faster to train. Region proposal is now the bottleneck.*

## Faster R-CNN

2015

Replace selective search with a Region Proposal Network that shares the backbone's features. Fully end-to-end.

---

*~5–7 fps and the accuracy standard for years. The base of Mask R-CNN.*

Feature Pyramid Network (FPN, 2017) — a top-down pathway with lateral connections that gives every scale a semantically strong feature map. It plugs into Faster R-CNN and most one-stage detectors alike, and is still standard practice.



# One-stage detectors: detection as regression

No proposal step. Predict class scores and box offsets directly from a dense grid of locations, in a single forward pass.



## YOLO

You Only Look Once, 2016

Divide the image into an  $S \times S$  grid; each cell predicts boxes, confidences and class probabilities in one pass.



## SSD

Single Shot Detector, 2016

Predict from several feature maps at different resolutions, using default boxes of several aspect ratios per location.



## RetinaNet

2017

One-stage accuracy had lagged because of background imbalance. Focal loss down-weights easy negatives and closes the gap.



## Anchor-free

FCOS, CenterNet, 2019

Drop anchor boxes entirely: predict object centres and distances to the box edges. Fewer hyper-parameters to tune.

The YOLO line keeps evolving

v3 (multi-scale) → v4/v5 (engineering and training tricks) → v7/v8 (anchor-free, decoupled heads) → v9–v12. Real-time, easy to train on custom data, and the usual first thing to try in production.



# The transformer era of detection

## DETR · Detection Transformer, 2020

Treats detection as direct set prediction. A CNN backbone feeds a transformer encoder–decoder, and a fixed set of learned object queries each emit one box and one class.

A bipartite matching loss assigns predictions to ground truth, so no anchor boxes and no NMS are needed at all.

### Which family should you pick?

Real-time video or edge device → a YOLO variant.

Maximum accuracy, offline batch → DETR-family or Faster R-CNN with FPN.

Novel classes with few labels → an open-vocabulary model.

## Deformable DETR

Attends to a sparse set of sampled points instead of the full feature map — fixes DETR's slow convergence and weak small-object performance.

## DINO / DINO-DETR

Improved denoising training and query selection; pushed DETR-style models to the top of the COCO leaderboard.

## RT-DETR

A real-time DETR variant that finally matches YOLO speed without needing NMS post-processing.

## Open-vocabulary detection

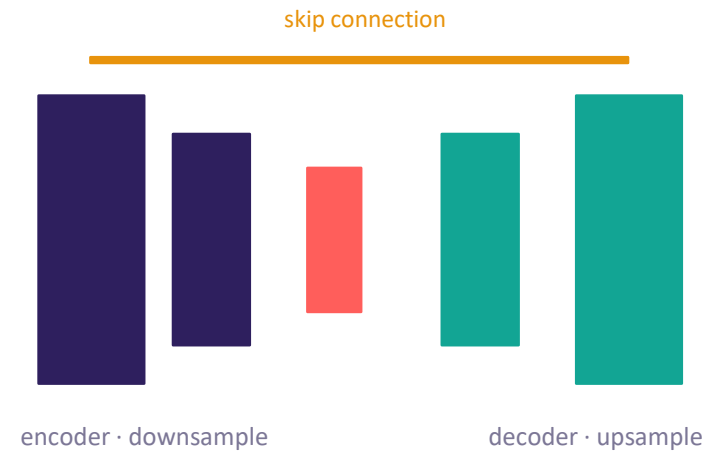
Grounding DINO, OWL-ViT and similar models detect classes described in free text, with no retraining for new categories.



# Semantic segmentation: a label for every pixel

- Assign each pixel a class. All cats share one label — the task does not distinguish instances.
- The naive approach, classifying a small patch around every pixel with a sliding window, is hopelessly expensive and ignores shared computation.
- The Fully Convolutional Network (FCN, 2015) replaced the dense layers with  $1 \times 1$  convolutions, so the network outputs a full-resolution class map in one pass.
- Downsampling with pooling and strided convolutions, then upsampling with transposed convolutions or interpolation, gives the encoder-decoder shape used everywhere since.
- Skip connections carry fine spatial detail from the encoder across to the decoder, recovering the boundaries that pooling destroyed.

## The encoder-decoder shape



## Metrics

Mean IoU over classes (the headline number) · pixel accuracy · Dice / F1 coefficient, common in medical imaging.



# The main semantic segmentation architectures

## FCN

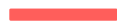
2015



The first end-to-end per-pixel network. Dense layers become convolutions; coarse predictions are upsampled and fused with earlier layers.

## U-Net

2015



Symmetric encoder–decoder with dense skip connections. Designed for biomedical images and still the default whenever data is scarce.

## SegNet

2016



Decoder upsamples using the max-pooling indices recorded by the encoder — memory-efficient, no learned upsampling parameters.

## DeepLab v3+

2018



Atrous (dilated) convolutions enlarge the receptive field without losing resolution; ASPP pools context at several dilation rates.

## PSPNet

2017



A pyramid pooling module aggregates global context at four scales before the final prediction.

## SegFormer / ViT-based

2021 →



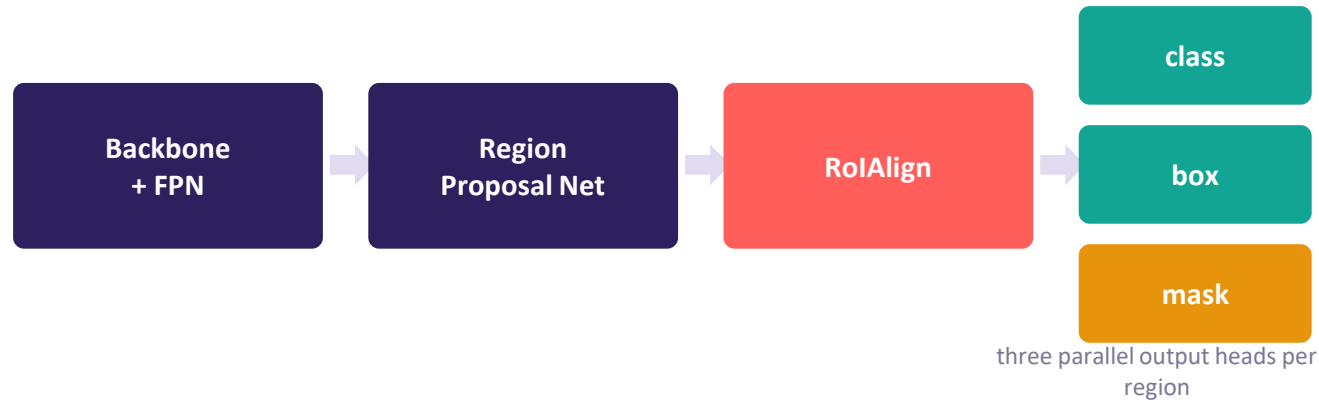
Transformer encoders with lightweight all-MLP decoders. Strong global context and excellent accuracy-per-FLOP.



# Instance segmentation: Mask R-CNN

Detect every object and produce a pixel mask for each one — so two overlapping dogs get two separate masks, not one merged region.

Faster R-CNN, plus one extra branch



## Why RoIAlign matters

RoI pooling rounds region coordinates to the feature-map grid. That misalignment is invisible for a box, but ruins a pixel mask.

RoIAlign uses bilinear interpolation and no rounding, which alone gave a large jump in mask accuracy.

- The mask head is a small FCN applied to each region, predicting a binary mask per class — the classification and mask tasks are decoupled.
- Swap the mask head for a keypoint head and the same architecture does human pose estimation.
- Still the standard baseline whenever accuracy matters more than speed.



# Where segmentation is heading

## YOLACT / SOLO

Real-time instance segmentation. YOLACT generates prototype masks and combines them with per-instance coefficients; SOLO predicts masks directly from location and size, with no detection step at all.

## Panoptic segmentation

Unifies the two views: every pixel gets a semantic label, and every countable object ('things') also gets an instance id, while amorphous regions ('stuff') do not.

## Mask2Former

One architecture — masked attention over learned queries — that handles semantic, instance and panoptic segmentation with the same model and loss.

## Segment Anything (SAM)

A promptable foundation model trained on a billion masks. Click a point or drag a box and it segments the object, zero-shot, with no task-specific training.



# One table to hold it all together

| Task                                                                                                                       | Output                              | Representative models                                             | Metric                        |
|----------------------------------------------------------------------------------------------------------------------------|-------------------------------------|-------------------------------------------------------------------|-------------------------------|
|  <b>Image classification</b>              | One class label                     | AlexNet · VGG · ResNet · EfficientNet · ViT                       | <b>Top-1 / top-5 accuracy</b> |
|  <b>Classification<br/>+ localisation</b> | One label + one box                 | ResNet backbone with a box-regression head                        | <b>Accuracy + IoU</b>         |
|  <b>Object detection</b>                  | N boxes + labels<br>+ scores        | Faster R-CNN · YOLO (v3 → v12) · SSD · RetinaNet · DETR · RT-DETR | <b>mAP@[.5:.95]</b>           |
|  <b>Semantic segmentation</b>             | Class label per pixel               | FCN · U-Net · SegNet · DeepLab v3+ · PSPNet · SegFormer           | <b>Mean IoU · Dice</b>        |
|  <b>Instance segmentation</b>           | Mask + label per object             | Mask R-CNN · YOLACT · SOLOv2 · Mask2Former · SAM                  | <b>Mask mAP</b>               |
|  <b>Panoptic segmentation</b>           | Semantic + instance,<br>every pixel | Panoptic FPN · Mask2Former                                        | <b>Panoptic Quality (PQ)</b>  |

*All of them share the same convolutional (or transformer) backbone — only the head and the loss change.*



## KEY TAKEAWAYS

# What to remember from today

- 01 Convolution is a constrained matrix multiply**

Sparse interactions, shared parameters and translation equivariance are the three constraints — and all three are assumptions about images, not tricks.
- 02 Shape arithmetic is the everyday skill**

$(N + 2P - K) / S + 1$  for the size,  $(K \times K \times D + 1) \times K_{\text{out}}$  for the parameters. Almost every debugging session comes back to these two.
- 03 Depth is the story of the architectures**

$8 \rightarrow 19 \rightarrow 22 \rightarrow 152$  layers. Small filters,  $1 \times 1$  bottlenecks and residual connections were each invented to make more depth trainable.
- 04 Fully connected layers are where the weights hide**

94 % of AlexNet and 90 % of VGG. Global average pooling removed them at no cost in accuracy.
- 05 Beyond classification, only the head changes**

Detection, semantic and instance segmentation all reuse the same backbone. Choose the model by the output you need and the latency you can afford.